
Testing Document

Team 09

2252709 Xuanhe Yang

2251548 Jingxiao Han

2251653 Tong Li

2253715 Fubin Chen

Testing document

1. Risk Analysis

1.1 Overview

We selected six dimensions for Risk Analysis:

- Functionality
- Performance
- Usability
- Security
- Reliability
- Maintainability

We identified a total of 33 risk points. Each risk can be traced back to specific requirements in the system design document and interface specification.

1.2 Risk Analysis Table (including demand risk mapping)

Requirements	Risk ID	Risk Category	Risk description	Risk Level	Impact level
User Login and Authentication Security	F-001	Functionality	Failure mechanism failure of user authentication JWT token	High	High
Device data upload and verification	F-002	Functionality	Improper verification of device data upload file format	Medium	Medium
Alarm management consistency	F-003	Functionality	Batch alert operation data inconsistent	High	High
Standard Data Management and Adaptation	F-004	Functionality	Standard data file is corrupted or incompatible	Extremely high	Extremely high
Device status and message synchronization	F-005	Functionality	Device status synchronization failed	Medium	Medium
High concurrency and scalability of the system	P-001	Performance	High system load due to large number of concurrent user accesses	High	High
Data Management and Index Optimization	P-002	Performance	Database query performance degradation	Medium	High
File I/O performance	P-003	Performance	Slow file upload and download	Medium	Medium

File, I/O performance	P-003	Performance	speed	medium	medium
Real - time refresh and visualization	P-004	Performance	Real - time monitoring data refresh delay	High	High
High usability and continuity requirements	U-001	Usability	System downtime or service interruption	Extremely high	Extremely high
High usability of API interface	U-002	Usability	API service unavailable	High	High
Database connection and backup	U-003	Usability	Database connection failed	Extremely high	Extremely high
External service integration usability	U-004	Usability	Third-party dependent service interruption	Medium	Medium
Network adaptation and switching capabilities	U-005	Usability	Network connection is unstable	Medium	Medium
Secure Input, API Parameterization	S-001	Security	SQL injection attack risk	Extremely high	Extremely high
User Authentication and Token Management	S-002	Security	JWT token leakage or forgery	Extremely high	Extremely high
Permission Model and RBAC	S-003	Security	Improper user rights control	High	High
HTTPS/Encrypted Transmission	S-004	Security	Sensitive data is transmitted without encryption	Extremely high	Extremely high
File upload security verification	S-005	Security	File upload malicious code injection	High	High
Front - end security and filtering verification	S-006	Security	XSS Cross Site Scripting	High	High
API Interface Authentication and Authorization	S-007	Security	Unauthorized access to API interface	High	High
Strong password policy	S-008	Security	Password policy is not strong enough	Medium	Medium
Automatic backup and recovery mechanism	R-001	Reliability	Data backup mechanism is not perfect	Extremely high	Extremely high
Fault recovery / disaster recovery capabilities	R-002	Reliability	Insufficient system fault recovery capabilities	High	High
Data Consistency/Byte Transaction	R-003	Reliability	Data consistency problem	High	High
Health Check and Exception Handling	R-004	Reliability	The exception handling mechanism is incomplete	Medium	Medium
Data quality inspection mechanism	R-005	Reliability	Sensor data quality issues	Medium	High
Architecture Design and Hierarchy	M-001	Maintainability	The code structure is complex and difficult to maintain	Medium	Medium
Log/Audit Log	M-002	Maintainability	Lack of complete logging	Medium	Medium
Configuration Center / Parameter Management	M-003	Maintainability	Improper configuration management	Low	Medium
Document Maintenance		Maintainability	The document is not updated in		

Document Maintenance Process	M-004	Maintainability	The document is not updated in time	Low	Low
System monitoring metrics/platform	M-005	Maintainability	Insufficient system monitoring capabilities	Medium	Medium
Version control / release mechanism	M-006	Maintainability	Version control and release process is not perfect	Medium	Medium

1.3 Risk level distribution statistics

Risk Level	Quantity	Proportion
Extremely high priority risk	7	16.70%
High-priority risk	16	38.10%
Medium priority risk	16	38.10%
Low-priority task risk	3	7.10%
Total	42	100%

1.4 Risk distribution by functional module

Function Module	Number of risk points	Proportion
Facility Management Module	8	19.00%
Alarm Management Module	7	16.70%
User Management Module	6	14.30%
Facility Monitoring Module	6	14.30%
Report Management Module	Five	11.90%
Standard Data Management Module	4 pieces	9.50%
Simulation Module	3	7.10%
System Infrastructure	3	7.10%
Total	42	100%

1.5 Detailed description of key risks

Extremely high priority risks (to be addressed immediately)

Risk number	Risk description	Impact description	Business consequences
F-004	Standard data file is corrupted or incompatible	May cause the entire prediction function to fail	Affect the core business and fail to predict the health of the device
U-001	System downtime or service interruption	may cause the entire platform to be unavailable	The service is completely interrupted, affecting device monitoring
U-003	Database connection failed	May cause all data operations to fail	Data read and write failed, system function is paralyzed
S-001	SQL injection attack risk	May lead to data leakage or system attacks	Sensitive data exposure threatens system security
S-002	JWT token leakage or forgery	May lead to unauthorized access and privilege bypass	Unauthorized users gain access to the system
S-004	Sensitive data is transmitted without encryption	May lead to data being stolen during transmission	Data intercepted during transmission
R-001	Data backup mechanism is not perfect	High risk of data loss	Permanent loss of critical business data

1.6 Risk Impact Analysis

1.6.1 Business Impact Assessment

1. High Impact Risk:

- Failure of the prediction function may lead to wrong maintenance decisions and increase the risk of equipment failure
- System unavailability will directly affect the continuity of aero-engine monitoring operations
- Data breaches can expose trade secrets and sensitive information
- Security bugs may bring legal compliance risks

2. Medium Impact Risk :

- Performance issues affect User Experience and work efficiency
- Data consistency issues can lead to decision - making biases
- Difficult system maintenance increases operating costs

1.6.2 Technology Impact Assessment

- **System stability:** Database failures, network outages, etc. may lead to overall system instability
- **Data integrity:** Backup mechanism defects and concurrent operation conflicts may damage data integrity
- **Security:** Multiple security bugs may be maliciously exploited, threatening system security

- **Scalability:** Architectural design flaws may limit the system's ability to scale in the future

1.7 Risk Response Strategy

1.7.1 Immediate response measures

1. Security hardening

- Immediately implement parameterized queries to prevent SQL injection
- Strengthen the security management of JWT tokens and rotate them regularly
- HTTPS encryption is enabled for the entire site
- Implement input validation and XSS protection

2. System Availability Assurance

- Establish automated backup and recovery mechanisms
- Implement database connection pooling and failover
- Add system health monitoring and alarm

1.7.2 Short-Term Improvement Plan

High - priority risk handling :

1. Standardization Verification and Control

- Standardize the interface & write automated scripts to automatically perform compatibility checks on uploaded standard data files, not only checking the format but also running test case samples.
- Introduce metadata such as version number, timestamp, upload user, and source, and all standard data files come with backup and version chain.
- The upload process goes through the pre-verification + pre-release environment:

2. Permission control optimization

- Implement fine grain role - based access control
- Establish user behavior audit log
- Add multi-factor authentication

3. System Monitoring Enhancement

- Deploy a real-time performance monitoring system
- Establish an anomaly detection and automatic alarm mechanism
- Implement centralized log management

4. Data Quantity Check

- Establish a sensor Data Quantity Check mechanism
- Implement data cleaning and outlier handling
- Add data consistency check

1.7.3 Medium and long - term plan

1. Architecture Upgrade

- Transform the service structure to improve system Modularization
- Containerized deployment to improve system portability
- Implement ByteMesh Management

2. Operation and Maintenance System Construction

- Establish DevOps process and automated deployment
- Improve disaster recovery and Business Continuity Plan
- Establish the operation and maintenance Knowledge Base and troubleshooting manual

1.8 Risk Monitoring and Early Warning

1.8.1 Monitoring Index System

1. Technical Specification:

- System response time: ≤ 3 seconds
- System availability: $\geq 99.9\%$
- Database connection success rate: $\geq 99.8\%$

2. Security Index:

- Number of security incidents: 0 times/month
- Authentication failure rate: $\leq 1\%$
- Abnormal Login Detection: Real - time Monitoring

3. Business Metrics:

- Customer Satisfaction Score: $\geq 90\%$
- Fault resolution time: ≤ 4 hours
- Data backup success rate: 100%

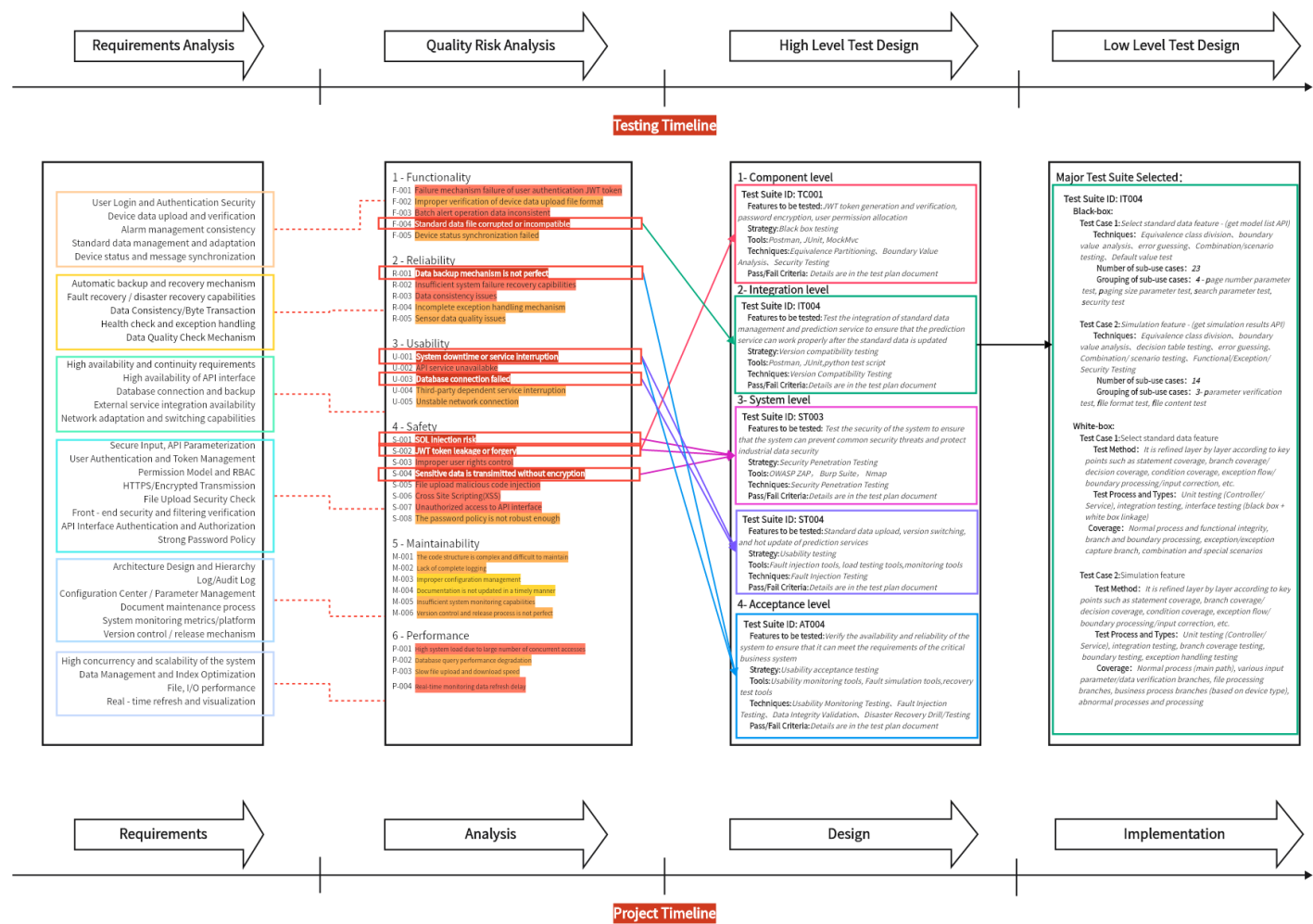
1.8.2 Early Warning Mechanism

- **Automated Monitoring:** 7×24 hours system health status monitoring
- **Threshold Alarm:** Automatically send an alarm when a key indicator exceeds the threshold

- **Trend Analysis:** Predict latent risk based on historical data
- **Emergency Response :** Establish a hierarchical response mechanism and a list of contacts

1.9 Requirement - Risk - High Level Test Suite - Low Level Test Design Mapping Diagram

According to the project design document, we extracted about 33 requirements, corresponding to the analysis of the risks in the project and the impact of the risk items. Based on these risks, in the test plan document, we designed a high-level test suite. For the main function of aeroengine life prediction, we refined and designed low-level test cases in the detailed design document. The following is the mapping diagram of requirements, risks, test suites and test cases. The picture shows some requirement information and the corresponding risk analysis items. We selected several extremely high priority risks, corresponding to the suite design covering these risk items and the detailed design of the underlying test cases.



1.10 Summary

This Risk Analysis Report comprehensively identifies 33 latent risks that the intelligent aero - engine industrial equipment maintenance platform may face in the production environment. Among them, the security and functional risks are the most prominent and need to be prioritized. Through systematic risk response strategies and continuous monitoring and early

warning mechanisms, system risks can be effectively reduced to ensure the stable operation of the platform.

2. Project Scope

2.1 Background

The Proactive Shield is an industrial health management system for aero - engines and their key components, aiming to provide users with **intelligent health assessment** , **anomaly monitoring** and **equipment remaining useful life assessment** services. With the continuous improvement of the requirements for equipment reliability and safety in the aviation industry, the traditional regular maintenance model can no longer meet the operation and maintenance needs of modern aero - engines. The platform performs **standard data files** collected during the operation of the equipment **real - time verification** , **processing** and **analysis** to achieve comprehensive monitoring of the engine operating status and data - driven operation and maintenance optimization.

The platform serves **equipment operators, maintenance engineers** and **system admins** . Equipment operators can monitor the equipment status and receive health warnings through the platform. Maintenance engineers can optimize maintenance plans based on the data reports and evaluation results provided by the platform. System admins are responsible for platform configuration, user management and system operation monitoring.

The core objective of Proactive Shield is to **conduct health assessment and risk determination of equipment based on standard data files** , thereby reducing unplanned downtime by 25 - 30%, lowering maintenance costs by 20 - 25%, increasing overall equipment efficiency by 15 percentage points, and ensuring an anomaly detection accuracy rate of over 95%.

2.2 Overall objective

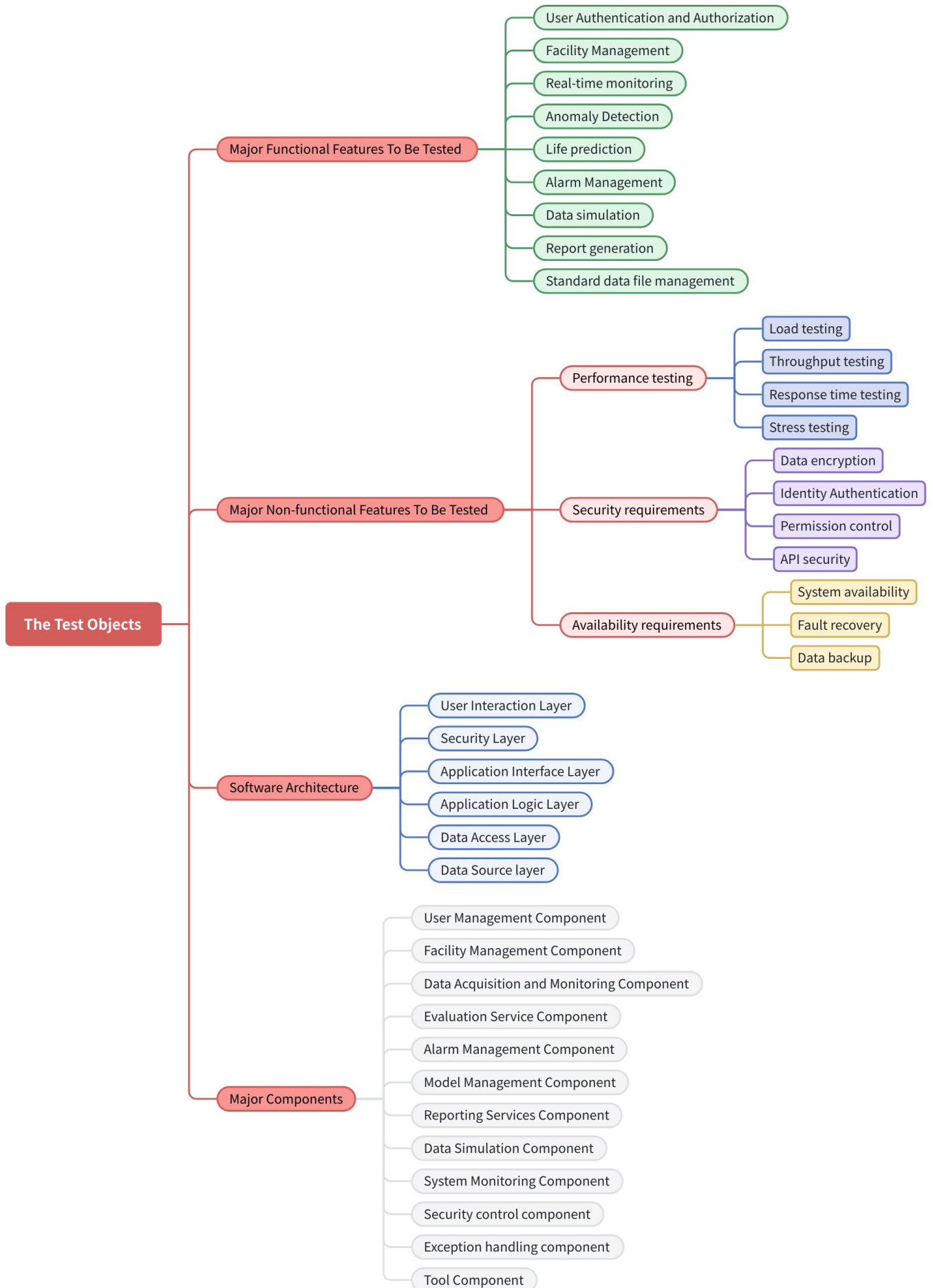
The overall objective of this project is to develop an **industrial equipment maintenance platform with high reliability, high availability, security and compliance** to meet the strict requirements of the aviation field for maintenance work and ensure that the platform can operate stably in a variety of industrial environments. The specific objectives are as follows:

1. **Functional integrity** : The platform should provide complete intelligent maintenance functions such as Facility Management, real-time monitoring, anomaly detection, life prediction, alarm management, and report generation.
 - Provide an intuitive monitoring interface for operators, including device status display, real-time data lake visualization, and early warning information reception.

- Provide professional analysis tools for maintenance engineers, including prediction result interpretation, maintenance advice generation, historical data analysis, etc.
 - Provide complete background management functions for system admins, including user management, device configuration, standard data file management, system monitoring, etc.
2. **High-precision prediction system:** Ensure that the accuracy of anomaly detection exceeds 95%, and the prediction error of remaining life is controlled within $\pm 10\%$, providing reliable data support for maintenance decision-making.
 3. **Security and Data Protection:** The platform must ensure the security of industrial data and standard data files, use multi - level encryption to protect sensitive information, and ensure compliance with aviation industry security standards.
 4. **Performance and real-time:** The platform needs to support real-time reception and processing of high-frequency sensor data. The evaluation and response time needs to be within the set threshold (such as 3 seconds) to ensure real-time and efficient monitoring and business collaboration .
 5. **User Experience:** Provide professional and user-friendly interface design for users in different roles to ensure that users can use the system efficiently in various working environments.
 6. **Scalability and Maintainability:** The system should have a good modular design, supporting flexible integration of new device types, data formats and business rules for iterative maintenance and function upgrade.

3. Test object

3.1 Overview of Test Objects



3.2 Non-functional characteristics

- **Performance Test:**

- **Load testing:** The platform needs to maintain stable performance in high-concurrency access scenarios and support more than 1,000 concurrent users to access at the same time.
- **Throughput testing :** The system needs to support the processing capacity of more than 1000 sensor Data Points per second to ensure that there is no delay in real-time data processing.
- **Response time test:** The API response time should be controlled within 1 second. The data processing and result return time of key business processes should not exceed 5 seconds.
- **Stress testing :** Verify the performance and recovery ability of the system under extreme load.

- **Security requirements:**

- **Data Encryption:** Sensitive data is encrypted and stored using AES-256, and TLS 1.3 is used for encryption during transmission.
- **Identity Authentication :** Support JWT token authentication and multi - factor authentication to ensure user identity security.
- **Permission Control :** Implement role - based fine grain permission control to prevent unauthorized operations.
- **API security:** Implement API limit viewership of, input validation, and SQL injection protection.

- **Availability Requirements :**

- **System availability:** The target system availability $\geq 99.9\%$
- **Fault recovery :** Key services can be automatically restored within 15 minutes after a failure
- **Data backup :** Implement automated data backup and disaster recovery mechanisms

4. High - level test suite design

4.1 Overview

The following is a rough mind map of a test suite:

Test Suites

Unit Level

- TC001:** Test user authentication functionality to ensure that users can securely log in to the system and obtain the correct JWT token
- TC002:** Test the Facility Management function to ensure that the device information can be correctly registered, updated and queried
- TC003:** Test the alarm management function to ensure that alarms can be correctly generated, processed and notified
- TC004:** Test data monitoring function to ensure that sensor data can be collected and visualized in real time
- TC005:** Test the report generation function to ensure that the system can generate accurate analysis reports
- TC006:** Test data simulation function to ensure that the simulation system can generate reasonable test data

Integration Level

- IT001:** Test the integration of the user authentication and permission control modules to ensure that users can correctly access authorized functions after logging in
- IT002:** Test the integration of the Facility Management and Data Acquisition modules to ensure that device data can be correctly transferred and displayed
- IT003:** Test the integration of the prediction service and the alarm system to ensure that the prediction results can correctly trigger the alarm
- IT004:** Test the integration of standard data management and prediction service to ensure that the prediction service can work properly after the standard data is updated
- IT005:** Integrate test data acquisition and report generation to ensure the accuracy and integrity of the data source for the report
- IT006:** Test the integration of system monitoring and exception handling to ensure that system exceptions can be detected and handled in a timely manner

System Level

- ST001:** Test the overall functionality of the system to ensure that all modules are correctly integrated and achieve the expected intelligent maintenance functionality in the system environment
- ST002:** Test the performance of the system under high concurrency and big data scenarios to ensure that the system can meet the requirements of industrial applications
- ST003:** Test the security of the system to ensure that the system can prevent common security threats and protect industrial data security
- ST004:** Test the high availability of the system to ensure that the system can meet the 99.9 % availability requirement
- ST005:** Test the scalability of the system to ensure that the system can adapt to the needs of business growth
- ST006:** Test the data integrity and consistency of the system to ensure the accuracy and reliability of industrial data
- ST007:** Test the operation and maintenance capabilities of the system to ensure that the system has good maintainability and monitorability

Acceptance Level

- AT001:** Ensure that the core functions of the system meet the requirements of intelligent maintenance business, effectively reduce maintenance costs and improve equipment efficiency
- AT002:** Verify the performance of the system in the production environment to ensure that it can meet the real-time requirements of industrial applications
- AT003:** Verify the security of the system to ensure compliance with the security compliance requirements of the aviation industry
- AT004:** Verify the availability and reliability of the system to ensure that it can meet the requirements of critical business systems
- AT005:** Verify that the system can be integrated with existing industrial systems to ensure the feasibility of deployment, operation and maintenance
- AT006:** Verify the User Experience of the system to ensure that users in different roles can use the system efficiently to complete their tasks
- AT007:** Verify the business value of the system to ensure that the expected maintenance cost reduction and efficiency improvement goals can be achieved

4.2 Test Suite Composition

4.2.1 Unit Level

Test Suite ID: TC001

Field	Content
Objective	Test the Facility Management function to ensure that device information can be correctly registered updated and queried
Feature to be tested	Device registration, device information update, device status management, device picture upload
Test Strategy	Functional testing: Verify the complete business process of Facility Management
Test Tool	Selenium, Postman, JUnit
Test Techniques	Equivalence Partitioning、 Boundary Value Analysis、 Security/Negative Testing
Risk Coverage	F-002 device data upload file format verification improper, F-005 device status synchronization failure, S-005 file upload malicious code injection
Passing Criteria	The device can be successfully registered and assigned a unique ID The device information can be correctly saved and displayed after modification File upload should verify format and size limits

Test Suite ID: TC002

Field	Content
Objective	Test the Facility Management function to ensure that device information can be correctly registered updated and queried
Feature to be tested	Device registration, device information update, device status management, device picture upload
Test Strategy	Functional testing: Verify the complete business process of Facility Management
Test Tool	Selenium, Postman, JUnit
Test Techniques	Equivalence Partitioning、 Boundary Value Analysis、 Security/Negative Testing
Risk Coverage	F-002 device data upload file format verification improper, F-005 device status synchronization failure, S-005 file upload malicious code injection
Passing Criteria	The device can be successfully registered and assigned a unique ID The device information can be correctly saved and displayed after modification File upload should verify format and size limits

Test Suite ID: TC003

Field	Content
Objectives	Test the alarm management function to ensure that alarms can be correctly generated, processed, and notified
Feature to be tested	Alarm generation, batch processing, status update, and export functions
Test Strategy	Integration testing: Test the integration of the alarm system with other modules
Test Tool	Selenium, Postman, JUnit
Testing Techniques	Equivalence Partitioning、 State Transition Testing、 Integration Testing / Data Consistency Check
Risk Coverage	F-003 Batch alarm operation data inconsistency, R-003 Data consistency issue, M-002 Lack of complete logging
Passing Criteria	Alarms should be generated correctly according to the pre - set rules Batch operations should ensure data consistency There should be a complete log record for alarm status change

Test Suite ID: TC004

Field	Content
Objectives	Test the data monitoring function to ensure that sensor data can be collected and visualized in real time
Features to be tested	Data Acquisition, Real-time Refresh, Chart Rendering, Historical Data Query
Test Strategy	Usability testing: Test the user experience and data display effect of the monitoring interface
Test Tool	Selenium, JMeter, browser power builder
Testing Techniques	Equivalence Partitioning、 Performance/Response Time Testing、 Usability Testing
Risk Coverage	P-004 Real-time monitoring data refresh delay, R-005 Sensor data quality issues, U-005 Unstable network connection
Pass Criteria	Data refresh delay should be ≤ 500ms The chart should correctly display real-time and historical data Appropriate error messages should be provided when network is abnormal

Test Suite ID: TC005

Field	Content
Target	Test the report generation function to ensure that the system can generate accurate analysis report
Features to be tested	Report template, data statistics, file export, and scheduled generation
Test Strategy	Data drive test (Data-driven testing): Use multiple sets of test data to verify the accuracy of the repo
Test tool	Postman, Excel comparison tool, JUnit
Testing Techniques	Data-driven Testing、 Equivalence Partitioning、 Negative/Exception Testing
Risk coverage	P - 003 file upload and download speed is slow, R - 004 exception handling mechanism is incomplete M - 004 document update is not timely
Passing criteria	The report data should be exactly the same as the source data The format of the exported file should be correct and can be opened normally Appropriate error handling should be provided for exceptional conditions

Test Suite ID: TC006

Field	Content
Objectives	Test data simulation function to ensure that the simulation system can generate reasonable test da
Feature to be tested	Parameter Configuration, Data Generation, Result Analysis, File Processing
Test Strategy	Boundary value testing: Test the system behavior under extreme parameters
Test Tool	Postman, Python Data Analysis Tools, JUnit
Testing Techniques	Boundary Value Analysis
Risk Coverage	Improper format verification of device data upload file for F-002, data quality issue of sensor for R-0 and complex and hard-to-maintain code structure for M-001
Passing Criteria	Simulation data should conform to physical laws and statistical distributions Extreme parameters should be correctly processed or rejected The generation results should be reproducible

4.2.2 Integration Level

The key points of integration testing include:

1. Interface test between modules: verify whether the REST API calls between different modules are correct, including data transmission, exception handling, etc.
2. Data Flow Integrity: Verify the complete data flow from sensor Data Acquisition to prediction result generation
3. Exception handling and fault tolerance mechanism: Ensure that the system can handle and recover correctly when a module fails
4. Real - time verification: Ensure that the real - time requirements can still be met after the integration of each module

Test Suite ID: IT001

Field	Content
Objectives	Test the integration of the user authentication and permission control module to ensure that users can correctly access the authorized functions after logging in
Feature to be tested	JWT token verification, role permission control, API access authorization
Test Strategy	Integration testing: verify the integration of the authentication module with the permission control of each business module
Test Tool	Postman, JUnit, MockMvc
Testing Techniques	Equivalence Partitioning、Integration Testing、Security/Negative Testing
Risk Coverage	S-003 improper user rights control, S-007 unauthorized access to API interface, F-001 failure of user authentication JWT token invalidation mechanism
Passing Criteria	Different roles of users should only be able to access authorized API endpoints Access should be properly denied when the token has expired Return appropriate HTTP status codes for authentication failures

Test Suite ID: IT002

Field	Content
Objective	Test the integration of the Facility Management and Data Monitoring modules to ensure that device data can be correctly transferred and displayed
Features to be tested	Data acquisition after device registration, data display on the monitoring interface, and device status synchronization
Test Strategy	Data flow testing: Track the complete path of data from acquisition to display
Test Tools	Selenium, Postman, Database Query Tool
Testing Techniques	Data Flow Testing
Risk Coverage	F-005 device status synchronization failure, P-004 real-time monitoring data refresh delay, R-005 sensor data quality issues
Passing Criteria	Newly registered devices should appear on the monitoring interface immediately The device data should be updated in real time and the data should be accurate The device should correctly display the status when it is offline

Test Suite ID: IT003

Field	Content
Objectives	Test the integration of the prediction service and the alarm system to ensure that the prediction results can correctly trigger the alarm
Feature to be tested	Prediction results processing, alarm rule triggering, and notification sending
Test Strategy	Event-driven testing: Test the complete process from the predicted event to the alarm generation
Test Tool	Postman, JUnit, email/SMS testing tools
Testing Techniques	Event-driven Testing
Risk coverage	F-003 Batch Alarm Operation Data Inconsistency
Passing Criteria	The abnormal prediction results should correctly trigger the corresponding level of alarm Alarm notifications should be sent to the designated recipients in a timely manner Batch prediction should be able to process correctly and generate corresponding alarms

Test Suite ID: IT004

Field	Content
Objective	Test the integration of standard data selection and simulation calculation functionality, ensuring that the simulation prediction function works properly when different standard data is selected
Features to be tested	Standard data list retrieval, parameter validation, simulation calculation, result presentation, exception handling
Test strategy	Black-box testing: equivalence partitioning, boundary value analysis, decision table method, error guessing White-box testing: statement coverage, branch coverage, path coverage
Test tools	JUnit 5, JaCoCo, Mockito, Spring test framework
Test techniques/methods	Unit testing, integration testing, exception injection, parameter validation testing, nested testing
Risk coverage	F-002: Improper verification of device data upload file format F-004: Standard data file is corrupted or incompatible S-001: SQL injection attack risk S-006: XSS Cross Site Scripting R-004: The exception handling mechanism is incomplete M-006: Version control and release process is not perfect
Pass criteria	1. Standard data list API can correctly handle pagination and search parameters, successfully returning data lists 2. Simulation calculation API can verify the format, structure, and content of uploaded files, rejecting invalid inputs 3. The system can generate accurate prediction results based on valid inputs, including remaining lifespan and health index 4. All error situations provide clear error messages and appropriate HTTP status codes 5. Unit tests and integration tests have a pass rate $\geq 90\%$, with code coverage for key components $\geq 80\%$

Test Suite ID: IT005

Field	Content
Objective	Integrate test data acquisition and report generation to ensure the accuracy and integrity of the data source for the report
Features to be tested	Data aggregation, statistical calculation, and consistency of report data
Test Strategy	Data consistency testing: Verify the consistency between the original data source and the report data
Test Tools	SQL Query Tool, Excel Comparison Tool, JUnit
Testing Techniques	Data Consistency Testing
Risk Coverage	R-003 data consistency issues, P-002 database query performance degradation, M-002 lack of complete logging
Passing Criteria	The Statistical Data in the report should be consistent with the original data source in the database The data aggregation process should have a complete audit log The performance of report generation should be acceptable in case of large data volume

Test Suite ID: IT006

Field	Content
Objectives	Test the integration of system monitoring and exception handling to ensure that system exceptions can be detected and handled in a timely manner
Feature to be tested	System health check, anomaly detection, automatic recovery mechanism
Test Strategy	Fault injection testing: Actively inject faults to test system responses
Test tool	System monitoring tools, fault injection tools, JUnit
Testing Techniques	Fault Injection Testing
Risk Coverage	U-002 API service unavailable, R-002 system fault recovery ability is insufficient, M-005 system monitoring ability is insufficient
Passing Criteria	The system component exception should be detected by the monitoring system in time When an exception occurs, the corresponding recovery mechanism should be triggered The recovery process should be well logged

4.2.3 System Level

The focus of system-level testing includes:

1. End - to - end functional verification: the complete business process from Data Acquisition to prediction result output
2. Non-functional requirements verification: key quality attributes such as performance, security, and usability
3. Industrial environment adaptability: stability and reliability in real industrial environments
4. Large-scale data processing capability: Verify the system's ability to process large amounts of sensor data

Test Suite ID: ST001

Field	Content
Objective	Test the overall functionality of the system to ensure that all modules are properly integrated in the system environment and achieve the expected intelligent maintenance functions
Feature to be tested	Facility Management, Real-time Monitoring, Anomaly Detection, Life Prediction, Alarm Management Report Generation
Test Strategy	E2E Test: Test the complete business process from data input to result output
Test Tool	Selenium, Postman, JMeter
Testing Techniques	End-to-end Testing
Risk Coverage	U-001 System unexpected downtime or service interruption, F-004 Standard data file corrupted or incompatible, R-003 Data consistency issue
Passing Criteria	The user can complete the whole process from device registration to getting the prediction result All modules of the system work together without any problems Data is transferred consistently between modules

Test Suite ID: ST002

Field	Content
Objective	Test the performance of the system under high concurrency and big data scenarios to ensure that the system can meet the requirements of industrial applications
Feature to be tested	Concurrent user processing, large sensor data processing, system response time
Test Strategy	Performance testing: Simulate high concurrency and big data scenarios to test system performance
Test Tool	JMeter, Gatling, System Monitoring Tools
Testing Techniques	Stress Testing
Risk Coverage	P-001 High system load due to large number of concurrent user access, P-002 Database query performance degradation
Passing Criteria	Support 1000+ concurrent users to access the system The overall system response time ≤ 3 seconds Process 1000+ Data Points per Second

Test Suite ID: ST003

Field	Content
Objectives	Test the security of the system to ensure that the system can prevent common security threats and protect industrial data security
Feature to be tested	SQL injection protection, Cross Site Scripting protection, JWT token security, data transmission encryption, permission control
Test Strategy	Security penetration testing: Simulate various security attacks to verify the system's protection capabilities
Test tool	OWASP ZAP, Burp Suite, Nmap
Testing Techniques	Security Penetration Testing
Risk coverage	S-001 SQL injection attack risk, S-002 JWT token leakage or forgery, S-004 sensitive data transmission without encryption, S-006 Cross Site Scripting attack
Pass criteria	The system can effectively prevent SQL injection attacks JWT token security mechanism is working properly Sensitive data is encrypted during the whole process of transmission Input validation can prevent XSS attacks

Test Suite ID: ST004

Field	Content
Objectives	Test the high availability of the system to ensure that the system can meet the 99.9% availability requirement
Feature to be tested	Failover, automatic recovery, data backup, load balance, health check
Test Strategy	Availability testing: Simulate various fault scenarios to test system availability
Test Tool	Fault injection tools, load testing tools, monitoring tools
Testing Techniques	Fault Injection Testing
Risk Coverage	U-001 System unexpected downtime or service interruption, U-003 Database connection failure, R-001 Data backup mechanism is not perfect
Passing Criteria	System annual availability $\geq 99.9\%$ Single point of failure does not affect the overall availability of the system Fault recovery time ≤ 15 minutes Data backup success rate 100%

Test Suite ID: ST005

Field	Content
Target	Test the scalability of the system to ensure that the system can meet the requirements of business growth
Features to test	Horizontal expansion, vertical expansion, microservice structure, containerized deployment
Test Strategy	Scalability testing: Verify the scalability of the system when the load increases
Test Tool	Kubernetes, Docker, load testing tools
Testing Techniques	Scalability Testing
Risk Coverage	P-001 High system load caused by a large number of concurrent user accesses, M-001 Complex code structure and difficult to maintain, M-006 Imperfect version control and release process
Pass Criteria	The system can handle load growth through horizontal scaling New service instances can be automatically added to the load balance The expansion process does not affect the use of existing users

Test Suite ID: ST006

Field	Content
Target	Test the data integrity and consistency of the system to ensure the accuracy and reliability of industrial data
Features to test	Data backup and recovery, transaction consistency, concurrency control, data validation
Test Strategy	Data integrity testing: Verify data integrity in various scenarios
Test Tool	Database testing tool, concurrent testing tool, data comparison tool
Testing Techniques	Data Integrity Testing
Risk Coverage	R-001 data backup mechanism is not perfect, R-003 data consistency problem, R-005 sensor data quality problem
Pass Criteria	Data backup and recovery functions are normal Concurrent operations do not destroy data consistency Data validation mechanism can detect quality problems

Test Suite ID: ST007

Field	Content
Target	Test the operation and maintenance capabilities of the system to ensure that the system has good maintainability and monitorability
Features to test	Log recording, performance monitoring, fault diagnosis, system configuration management
Test Strategy	Maintainability testing: Verify the operation and maintenance capabilities of the system
Test Tool	Log analysis tool, monitoring platform, configuration management tool
Testing Techniques	Maintainability Testing
Risk Coverage	M-002 lacks complete log records, M-005 has insufficient system monitoring capabilities, and M-003 has non-standard configuration management
Pass Criteria	The system operation and exception have complete log records The monitoring system can detect performance problems in time Configuration changes have full version control

4.2.4 Acceptance Level

The focus of acceptance testing includes:

1. Business Requirements Completeness: Verify that the system meets all business requirements for intelligent maintenance
2. User Experience Validation: Ensure that users in different roles can use the system efficiently
3. Industrial environment adaptability: verify the stability of the system in the real industrial environment
4. ROI Achievement: Verify whether the system can achieve the expected goals of reducing maintenance costs and improving efficiency

Test Suite ID: AT001

Field	Content
Target	Ensure that the core functions of the system meet the requirements of intelligent maintenance business, effectively reduce maintenance costs and improve equipment efficiency
Features to test	Anomaly Detection Accuracy, Life Prediction Accuracy, Maintenance Recommendation Quality, User Workflow
Test Strategy	User acceptance testing: Invite actual users to verify system functionality and usability
Test Tool	Real data sets, user feedback tools, business metrics monitoring
Testing Techniques	Equivalence Partitioning、Data-driven Testing、Cause-Effect Graphing、Scenario/Use-case Testing
Risk Coverage	F-004 Standard data file is corrupted or incompatible, U-004 Third-party dependent service is interrupted
Pass Criteria	Anomaly detection accuracy $\geq 95\%$ Life prediction error $\leq \pm 10\%$ The user can complete the whole maintenance decision - making process Customer Satisfaction Score $\geq 90\%$

Test Suite ID: AT002

Field	Content
Target	Verify the performance of the system in the production environment to ensure that it can meet the real-time requirements of industrial applications
Features to test	Real - time data processing, large - scale concurrency, long - term stable operation, resource utilization
Test Strategy	Production environment performance acceptance testing: verify performance indicators in a real environment
Test Tool	Production monitoring tools, performance benchmarking tools, resource monitoring tools
Testing Techniques	Benchmark Testing、Concurrency/Stress Testing、Stability/Soak Testing、Resource Utilization Analysis
Risk Coverage	P-001 The system load is too high due to a large number of concurrent user accesses, P-004 The real time monitoring data refresh is delayed
Pass Criteria	7×24 hours stable operation without major failures Support 1000+ concurrent users CPU and memory utilization are within reasonable limits

Test Suite ID: AT003

Field	Content
Target	Verify the security of the system to ensure compliance with the security compliance requirements of the aviation industry
Features to test	Data security, access control, audit tracking, compliance check
Test Strategy	Security compliance acceptance testing: Verify the effectiveness of system security controls
Test Tool	Security scanning tools, compliance checking tools, auditing tools
Testing Techniques	Penetration Testing、Access Control Testing、Audit Log Verification、Compliance Testing
Risk Coverage	S-001 SQL injection attack risk, S-003 improper user rights control, S-004 sensitive data transfer without encryption
Pass Criteria	Pass the security compliance check No high-risk security bugs Access control mechanism is effective Audit logs are complete and traceable

Test Suite ID: AT004

Field	Content
Target	Verify the availability and reliability of the system to ensure that it can meet the requirements of the critical business system
Features to test	System availability, fault recovery, data integrity, disaster recovery
Test Strategy	Reliability acceptance testing: Long - term operation test to verify system reliability
Test Tool	Availability monitoring tools, fault simulation tools, recovery test tools
Testing Techniques	Availability Monitoring Testing、Fault Injection Testing、Data Integrity Validation、Disaster Recovery Drill/Testing
Risk Coverage	U-001 System unexpected downtime or service interruption, R-001 Data backup mechanism is not perfect, R-002 System fault recovery ability is insufficient
Pass Criteria	The system annual availability rate $\geq 99.9\%$ Fault recovery time ≤ 15 minutes Zero data loss Disaster recovery drill successful

Test Suite ID: AT005

Field	Content
Target	Verify that the system can be integrated with existing industrial systems to ensure the feasibility of deployment, operation and maintenance
Features to test	System integration, data migration, deployment process, operation and maintenance support
Test Strategy	Integration acceptance testing: Verify the ability to integrate with existing systems
Test Tool	Integration testing tools, data migration tools, deployment automation tools
Testing Techniques	Interface Testing、Data Consistency Verification、Automated Deployment Testing、Operations Runbook Drill
Risk Coverage	U-004 Third-party dependent service interruption, M-006 Imperfect version control and release process, M-003 Irregular configuration management
Pass Criteria	Can be successfully integrated with existing monitoring systems Historical data migration is complete and error-free Automated and repeatable deployment process The operation and maintenance documents are complete and easy to understand

Test Suite ID: AT006

Field	Content
Target	Verify the User Experience of the system to ensure that users in different roles can use the system efficiently to complete their work tasks
Features to test	Interface friendliness, convenient operation, learning cost, work efficiency improvement
Test Strategy	User Experience Acceptance Testing: Real users use the system to complete real tasks
Test Tool	User behavior analysis tool, interface testing tool, efficiency statistics tool
Testing Techniques	Usability Testing、 Usability Testing、 Usability Testing、 Usability & Efficiency Analysis
Risk Coverage	M-004 document update is not timely, F-005 device status synchronization fails, P-003 file upload and download speed is slow
Pass Criteria	New users can master basic operations within 2 hours The operation steps of common functions are ≤ 3 steps User work efficiency is improved by $\geq 20\%$ compared with the traditional way User feedback satisfaction $\geq 85\%$

Test Suite ID: AT007

Field	Content
Target	Verify the business value of the system to ensure that the expected maintenance cost reduction and efficiency improvement goals can be achieved
Features to test	Maintenance cost reduction, equipment efficiency improvement, fault prediction accuracy, decision support quality
Test Strategy	Business value acceptance testing: Quantitatively evaluate the business value of the system
Test Tool	ROI calculation tool, business indicator analysis tool, comparative analysis tool
Testing Techniques	Business Metrics Comparison/Analysis、 Business Metrics Comparison/Analysis、 Data-driven Testing、 Real-world Scenario Replay
Risk Coverage	F-004 standard data file corrupted or incompatible, U-001 system unexpected downtime or service interruption
Pass Criteria	Unplanned downtime reduced by $\geq 25\%$ Maintenance cost reduction $\geq 20\%$ Overall equipment efficiency improvement $\geq 15\%$ Prediction accuracy $\geq 95\%$ Return on investment cycle ≤ 24 months

5. Schedule

4.1 Test Level

There are four levels of test plans for this project:

- Unit level
- Integration Level
- System level

- Acceptance level

4.2 Test Targets

Based on the designed high-level test suite, we have the following summary of test objectives:

Test Level	Test Objectives	No.	Details
Unit Test Level	functionality	TC001-TC007	Test whether the functionality of each component unit of the testing system is fully implemented.
Integration Test Level	functionality	IT001-IT006	Test whether the integrated functionality of multiple modules in the testing system operates as expected.
System Test Level	functionality	ST001	Test the overall functionality of the target testing system.
	Performance	ST002	Test the performance of the target testing system under high concurrency and large data volume scenarios to ensure the system meets industrial-grade application requirements.
	security	ST003	Test the security of the target testing system to ensure it can defend against common security threats and protect industrial data safety.
	usability	ST004	Test the high availability of the system to ensure it meets the 99.9% availability requirement.
	Scalability	ST005	Test the scalability of the system to ensure it can adapt to growing business demands.
	reliability	ST006	Test the data integrity and consistency of the system to ensure the accuracy and reliability of industrial data.
	maintainability	ST007	Test the system's operational and maintenance capabilities to ensure it has good maintainability and monitorability.
Acceptance Test Level	functionality	AT001	Verify that the core functionalities of the system meet the business requirements for

			intelligent maintenance.
	Performance	AT002	Verify the system's performance in the production environment to ensure it meets the real-time requirements of industrial-grade applications.
	security	AT003	Verify the system's security to ensure it complies with the safety and regulatory requirements of the aerospace industry.
	usability	AT004	Verify the system's availability and reliability to ensure it meets the requirements of critical business systems.
	reliability	AT005	Verify that the system can integrate with existing industrial systems to ensure the feasibility of deployment and maintenance.
	usability	AT006	Verify the user experience of the system.
	maintainability	AT007	Verify the business value of the system to ensure it achieves the expected reduction in maintenance costs.

6. Test Framework

6.1 Functional Test Framework

During the functional testing process, we adopted a variety of mainstream testing frameworks and tools, from unit testing, integration testing, system testing to acceptance testing, to ensure the full coverage and verification of system functions. The following is an analysis of the main testing frameworks used and their advantages:

1. JUnit 5

JUnit 5 is a modern Java unit test framework widely used for automated testing of small code blocks. Its features include:

- **Rich annotation support:** such as `@Test` , `@BeforeEach` , `@AfterEach` , etc., to improve the clarity and readability of the test structure
- **Strong extensibility:** Support use `@ExtendWith` to integrate with Spring and other frameworks
- **Flexible test organization** : The execution order of test methods can be controlled through `@TestMethodOrder`

- **Parameterized test:** Support `@ParameterizedTest` to implement data-driven test
- **Nested test structure :** Use `@Nested` to implement a clearer test hierarchy

2. Spring Test

Spring Test is a testing framework designed specifically for Spring applications, supporting everything from unit tests to full integration tests. The main advantages include:

- **Full integration test support:** `@SpringBootTest` can start the whole Spring Boot application for testing
- **Controller layer test optimization:** `@WebMvcTest` is used to isolate and test MVC controller logic
- **Easy to configure simulation environment :** `@AutoConfigureMockMvc` provides the ability to mock HTTP requests
- **Flexible environment adaptation :** Support different environment configuration files to facilitate multi-scenario testing
- **Dependency injection support :** Spring managed beans can be directly injected into the test

3. Mockito

Mockito is one of the most commonly used simulation frameworks in the Java field, which is used to isolate the dependencies of test objects and improve the controllability and stability of tests. Its advantages include:

- **Concise syntax:** Quickly create mock objects through `@Mock` , `@MockBean`
- **Automatic injection ability :** `@InjectMocks` can inject mock objects into the object under test
- **Behavior verification support:** Use `when()` and `verify()` to implement simulation logic and call verification
- **Integrates well with Spring:** Can be used seamlessly with Spring Test
- **Flexible simulation behavior configuration :** Support complex stub logic settings and exception simulation

4. Selenium WebDriver

Selenium is an automated testing framework for web applications that can simulate the user's operation process in the browser. Its main advantages are:

- **Cross-browser support:** Compatible with multiple browsers to ensure consistent functionality
- **Real user behavior simulation :** Automatically perform user operations such as form input, click, navigation, etc.

- **E2E Test Capability** : Cover whole-link test from login to business process
- **Rich API**: Provide detailed page element operation interfaces, suitable for complex UI testing

5. Postman

Postman is a powerful API development and testing tool widely used for interface debugging, automated testing, and interface documentation management. In this project, we used Postman to verify and functionally test the backend interfaces. Its main advantages include:

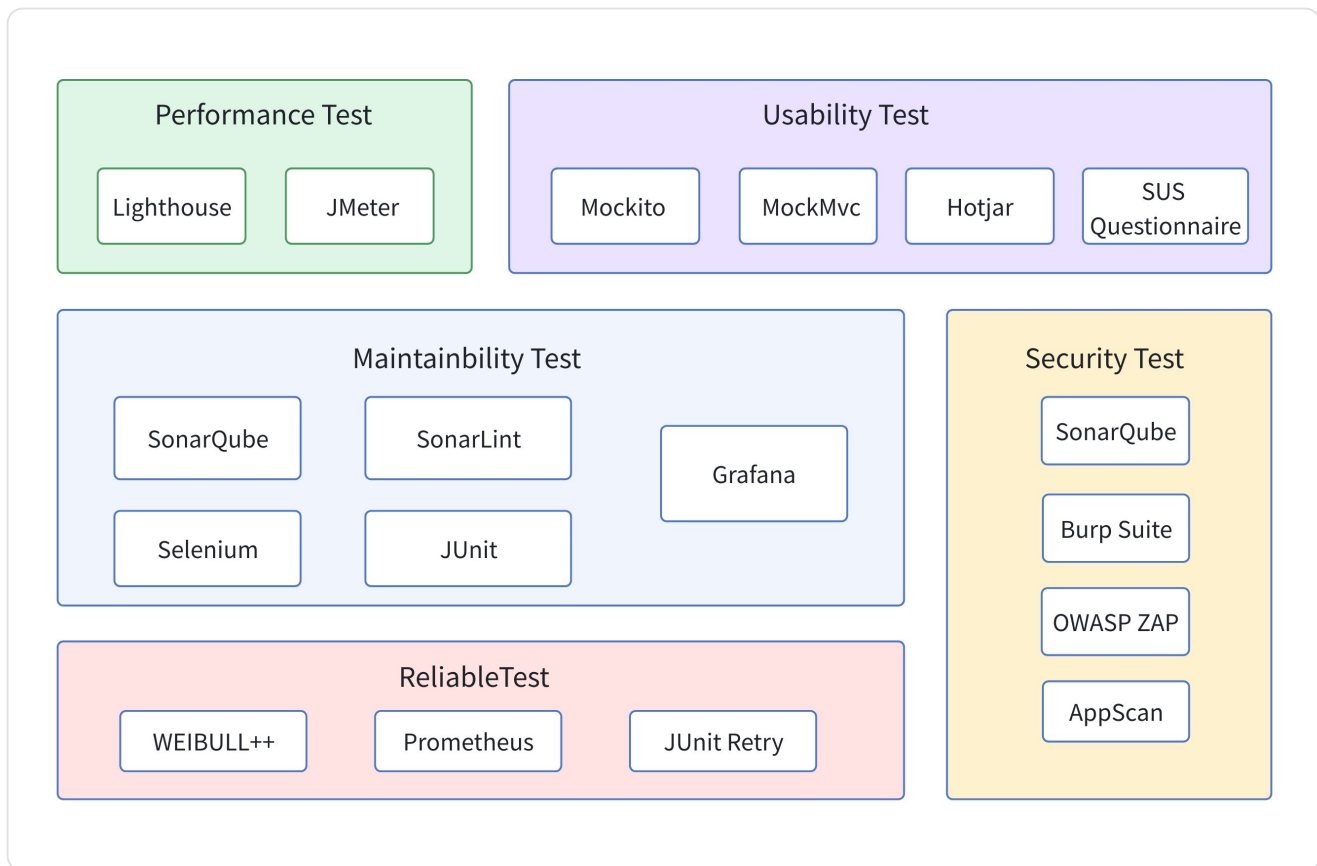
- **Support automated test sets** : Create request sets and write test scripts to verify multiple interfaces in batches
- **Environment variable management** : Support multi - environment configuration to facilitate the switching of interfaces between development, testing and production environments
- **Rich assertion function**: Write assertion logic through JavaScript to automatically verify response content, status code, etc.
- **Integratable with continuous integration**: Supports the CLI tool Newman and can be embedded in the CI/CD process to implement automated interface testing

Test Tool	advangtagement
JUnit 5	• Rich annotation support: Such as @Test, @BeforeEach, @AfterEach, etc., which improve the clarity and readability of test structures.
	• High extensibility: Supports integration with frameworks like Spring using @ExtendWith.
	• Flexible test organization: The execution order of test methods can be controlled by @TestMethodOrder.
	• Parameterized tests: Supports @ParameterizedTest for data-driven testing.
	• Nested test structures: Provides clearer test hierarchies using @Nested.
Spring Test	• Complete integration testing support: @SpringBootTest can start the entire Spring Boot application for testing.
	• Controller layer testing optimization: @WebMvcTest isolates and tests MVC controller logic.
	• Convenient mock environment configuration: @AutoConfigureMockMvc provides the ability to mock HTTP requests.

	• Flexible environment adaptation: Supports different environment configuration files, making it easy to test multiple scenarios. • Dependency injection support: Beans managed by Spring can be directly injected in tests.
Mockito	• Simple syntax: Quickly create mock objects using <code>@Mock</code>, <code>@MockBean</code>. • Automatic injection capability: <code>@InjectMocks</code> can inject mock objects into the object under test. • Behavior verification support: Uses <code>when()</code> and <code>verify()</code> to mock logic and verify method calls. • Good integration with Spring: Seamlessly integrates with Spring Test. • Flexible behavior configuration: Supports complex stubbing logic and exception simulation.
Selenium WebDriver	• Cross-browser support: Compatible with various browsers, ensuring functionality consistency. • Real user behavior simulation: Automatically performs form inputs, clicks, navigation, and other user actions. • End-to-end testing capability: Covers the full testing lifecycle, from login to business process workflows. • Rich API: Provides detailed interfaces for interacting with page elements, making it suitable for complex UI testing.
Postman	• Supports automated test collections: Allows the creation of request collections and writing test scripts to validate APIs in bulk. • Environment variable management: Supports multiple environment configurations, making it easy to switch between development, testing, and production environments. • Rich assertion features: Enables automatic validation of response content, status codes, etc., using JavaScript-based assertions. • Integration with continuous integration: Supports the CLI tool Newman, which can be embedded into CI/CD workflows for automated API testing.

6.2 Non-functional testing framework

The following figure shows the test framework for the non-functional testing of the project we designed:



1. Performance Test

Performance testing is mainly used to verify the system's responsiveness, stability, and throughput under different loads. We use the following tools:

- **JMeter**

Function Introduction: JMeter is an open-source load testing tool widely used for performance testing, load testing, and stress testing. It can simulate different user scenarios and generate detailed test reports.

- **Lighthouse**

Function Introduction: Lighthouse is an automated open-source tool for analyzing web performance. It helps developers identify potential performance bottlenecks and provides optimization suggestions.

2. Usability Test

Usability testing is used to evaluate the usability and User Experience of the system. We use the following tools:

- **Mockito**

Function Introduction: Tencent Meeting is an online communication platform for user testing. It can record meetings for subsequent analysis of user feedback.

- **MockMvc**

Function Introduction: Screen recording software is used to record the user's operation process, helping to analyze the user's behavior and problems when interacting with the system.

- **SUS Questionnaire**

Function Introduction: The SUS (System Usability Scale) questionnaire is a standard usability evaluation method that surveys users' satisfaction with the system through a questionnaire.

- **Hotjar**

Function Introduction: Hotjar provides functions such as heat map and user behavior analysis to collect user interaction data on web pages and help identify usability issues.

3. Security Test

Security testing ensures the system's resistance to external attacks. We use the following tools:

- **SonarQube**

Function Introduction: SonarQube is a static code analysis tool that detects security bugs, technical debt, and quality issues in code and supports multiple programming languages.

- **Burp Suite**

Function Introduction: Burp Suite is an integrated Web security testing tool that provides complete functions from scanning, attacking to analysis.

- **OWASP ZAP**

Function Introduction: OWASP ZAP (Zed Attack Proxy) is an open source security vulnerability scanning tool that helps identify security bugs in web applications.

- **AppScan**

Function Introduction: AppScan is an application security testing tool provided by IBM, focusing on identifying security bugs in web applications.

- **SonarLint**

Function Introduction: SonarLint is a real-time code quality plugin for SonarQube that can detect potential code defects and security bugs during development.

4. Reliability Test

To evaluate the stability of the reliability testing system under long - term operation, we use the following tools:

- **WEIBULL++**

Function Introduction: WEIBULL++ is a reliability analysis tool widely used for reliability testing in the product lifecycle. It conducts Data Analysis based on the Weibull distribution.

- **Prometheus**

Function Introduction: Prometheus is an open source system monitoring and alerting toolkit that can collect and store time series data and is suitable for real-time monitoring of highly reliable systems.

- **JUnit Retry Mechanism**

Function Introduction: Determine the stability of the system under critical conditions by repeatedly executing unit tests.

5. Maintainability Test

Maintainability testing is used to verify the maintainability of the system over a long period of time. We use the following tools:

- **SonarQube**

Function Introduction: Similarly, SonarQube is used for code quality analysis to help development teams improve system maintainability.

- **SonarLint**

Function Introduction: As a real-time code quality plug-in for SonarQube, SonarLint can detect potential code defects and security issues during the development process.

- **Selenium**

Function Introduction : Selenium is an automated testing tool for regression testing of web applications and can help detect the impact of system updates and patches.

- **JUnit**

Function Introduction: JUnit is a unit test framework for Java projects that can verify the functional integrity of the code and is a basic tool for maintaining automated tests.

- **Grafana**

Function Introduction : Grafana is an open - source monitoring tool that can be integrated with data sources such as Prometheus to provide beautiful real - time monitoring panels.

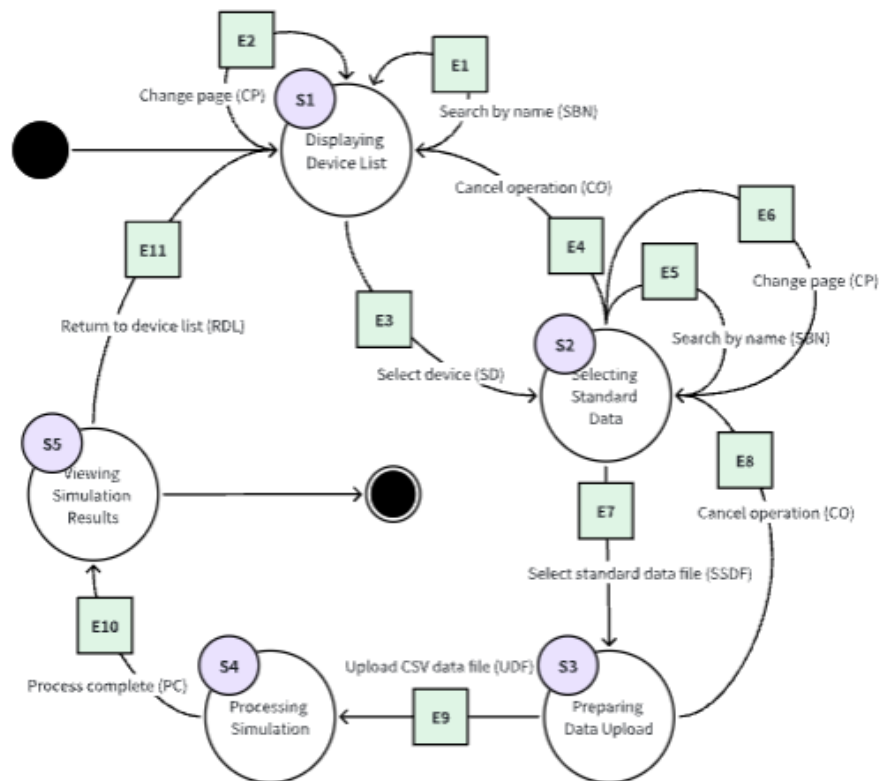
7. Test case design

The aero-engine life prediction module uses predictive analytics to help users estimate the remaining useful life of aircraft engines. The system workflow is as follows:

1. **Facility Management:** The user browses the equipment list and selects the engine to be analyzed.
2. **Standard Data Selection:** Select the appropriate standard reference data file for the selected engine.

3. **Parameter Upload:** Upload a CSV file containing the current engine operating parameters.
4. **Simulation processing:** The system compares and calculates the uploaded data with the standard model.
5. **Result analysis:** The user obtains analysis results including remaining life, health index and potential damage location.

This function helps the maintenance team optimize the engine maintenance plan and prevent unexpected failures.



For the main function of aero-engine life prediction, the following test design is made:

7.1 Black box test cases

7.1.1 Select the standard data function (Get standard data list API)

Parameter table

parameter	parameter description	Parameter required
page	Page number, default is 1	No
pageSize	Number of items per page, default is 10	No
searchQuery	Search keywords	No

A. Equivalence Class

page parameter

Type	Value	Description
Valid	1~N (within the total number of pages)	Valid page number
Valid	Not passed	Do not pass page number
Invalid	Non-positive number	Invalid page number value
Invalid	>N (beyond total pages)	Page number exceeds total number of pages
Invalid	Non - integer	Non - integer page number

pageSize parameter

Type	Value	Description
Valid	1~50	Reasonable number of items per page
Valid	Not passed	Do not pass the number of items per page
Invalid	Non-positive number	Invalid number of items per page
Invalid	>50	Too large number of items per page
Invalid	Non - integer	Non - integer page size

searchQuery parameter

Type	Value	Description
Valid	Arbitrary string	Keywords for fuzzy matching standard data names
Valid	Not passed	Do not perform search filtering and display all standard data
Invalid	Injection attack string	Strings containing SQL injection or other attack code, such as <code>'OR 1=1--</code> , <code><script>alert(1)</script></code>

B. Boundary value

page parameter

Number	Equivalent class	Boundary value	Description
BV-P-01	Valid (in the range of 1 to N)	1	Minimum effective page number
BV-P-02	Valid (in the range of 1 to N)	N (assumed to be 5)	Maximum effective page number (determined by the amount of data)
BV-P-03	Effective (not passed)	No parameters are passed	If the page parameter is not provided, the default value 1 should be used
BV-P-04	Invalid (non-positive)	0	Invalid page number (value 0)
BV-P-05	Invalid (non-positive)	-1	Invalid page number (negative number)
BV-P-06	Invalid (beyond total pages)	N+1 (assumed to be 6)	Invalid page number (exceeds the total number of pages by 1 unit)
BV-P-07	Invalid (beyond total pages)	999	Invalid page number (far exceeds total number of pages)
BV-P-08	Invalid (not an integer)	1.5	Invalid page number (decimal)
BV-P-09	Invalid (not an integer)	"abc"	Invalid page number (string)

pageSize parameter

Number	Equivalent class	Boundary value	Description
BV-PS-01	Valid (in the range of 1 to 50)	1	Minimum effective page size
BV-PS-02	Valid (in the range of 1 to 50)	50	Maximum effective page size
BV-PS-03	Effective (not passed)	No parameters are passed	If the pageSize parameter is not provided, the default value of 10 should be used

BV-PS-04	Invalid (non-positive)	0	Invalid page size (value 0)
BV-PS-05	Invalid (non-positive)	-1	Invalid page size (negative number)
BV-PS-06	Invalid (>50)	51	Invalid page size (1 unit above maximum)
BV-PS-07	Invalid (>50)	1000	Invalid page size (far exceeds maximum)
BV-PS-08	Invalid (not an integer)	10.5	Invalid page size (decimal)
BV-PS-09	Invalid (not an integer)	"abc"	Invalid page size (string)

searchQuery parameter

Number	Equivalent class	Boundary value	Description
BV-SQ-01	Valid (any string)	"fan"	Short English keywords (common matching words)
BV-SQ-02	Valid (any string)	"Turbine"	English keyword
BV-SQ-03	Valid (any string)	"a very long search string with many words"	Long keyword (test length limit)
BV-SQ-04	Valid (any string)	"Special characters!@#\$\$%"	Keywords containing special characters
BV-SQ-05	Valid (not passed or empty string)	""	empty string
BV-SQ-06	Valid (not passed or empty string)	No parameters are passed	If the searchQuery parameter is not provided, all data should be displayed
BV-SQ-07	Invalid (injection attack)	"" OR 1=1--"	SQL injection attack string
BV-SQ-08	Invalid (injection attack)	""	XSS attack string
BV-SQ-09	Invalid (injection attack)	"1; DROP TABLE users;"	Destructive SQL injection string

C. Test case design

API test case for obtaining the standard data list

Field	Content
Identification	TC_PROCESS_DATA_API_001
Test items	Basic function test of API for obtaining standard data file list
Input specifications	Test multiple input parameter combinations
Output specifications	Return the corresponding standard data file list data according to different input parameters
Environmental needs	JUnit testing environment
Special procedural requirements	A valid authentication token is required The number of data in the list exceeds the maximum number of pages
Inter-case dependencies	None

Sub Use Case

Identification	Test items	Input specifications	Output specifications
TC_PROCESS_DATA_API_001_01	BV-P-03, BV-PS-03, BV-SQ-06	<code>{}</code>	status: 200 OK The default values should be used: page=1, pageSize=10 Return the standard data file data on page 1, 10 items per page
TC_PROCESS_DATA_API_001_02	BV-P-01, BV-PS-01	<code>{"page": 1, "pageSize": 1}</code>	status: 200 OK Return the data on page 1, with 1 record per page
TC_PROCESS_DATA_API_001_03	BV-P-02, BV-PS-02	<code>{"page": 5, "pageSize": 50}</code>	status: 200 OK Return the data on page 5 with 50 records per page
TC_PROCESS_DATA_API_001_04	BV-P-04	<code>{"page": 0}</code>	Should return an error response or automatically correct to page=1
TC_PROCESS_DATA_API_001_05	BV-P-05	<code>{"page": -1}</code>	Should return an error response or automatically correct to

			page=1
TC_PROCESS_DATA_API_001_06	BV-P-06	<code>{"page": 6}</code>	If the total number of pages is 5, an empty list or error message should be returned
TC_PROCESS_DATA_API_001_07	BV-P-07	<code>{"page": 999}</code>	Should return an empty list or appropriate error message
TC_PROCESS_DATA_API_001_08	BV-P-08	<code>{"page": 1.5}</code>	Should return an error response or automatically correct to a valid integer value
TC_PROCESS_DATA_API_001_09	BV-P-09	<code>{"page": "abc"}</code>	Should return an error response or automatically correct to the default value
TC_PROCESS_DATA_API_001_10	BV-PS-04	<code>{"pageSize": 0}</code>	Should return an error response or automatically correct to the default value of 10
TC_PROCESS_DATA_API_001_11	BV-PS-05	<code>{"pageSize": -1}</code>	Should return an error response or automatically correct to the default value of 10
TC_PROCESS_DATA_API_001_12	BV-PS-06	<code>{"pageSize": 51}</code>	should return an error response or automatically correct to the maximum allowed value of 50
TC_PROCESS_DATA_API_001_13	BV-PS-07	<code>{"pageSize": 1000}</code>	should return an error response or automatically correct to the maximum allowed value of 50
TC_PROCESS_DATA_API_001_14	BV-PS-08	<code>{"pageSize": 10.5}</code>	Should return an error response or automatically correct to a valid integer value
TC_PROCESS_DATA_API_001_15	BV-PS-09	<code>{"pageSize": "abc"}</code>	Should return an error response or automatically correct to the default value of 10
TC_PROCESS_DATA_API_001_16	BV-SQ-01	<code>{"searchQuery": "fan"}</code>	status: 200 OK Return a list of standard data files whose names contain "fan".
TC_PROCESS_DATA_API_001_17	BV-SQ-02	<code>{"searchQuery": "涡轮"}</code>	status: 200 OK Returns a list of standard data files whose names contain

			"turbine".
TC_PROCESS_DATA_API_001_18	BV-SQ-03	<code>{"searchQuery": "非常长的搜索字符串包含多个词语用于测试系统对长文本的处理能力"}</code>	status: 200 OK The system should handle long search terms correctly
TC_PROCESS_DATA_API_001_19	BV-SQ-04	<code>{"searchQuery": "特殊字符!@#\$%"} </code>	status: 200 OK The system should handle special characters correctly
TC_PROCESS_DATA_API_001_20	BV-SQ-05	<code>{"searchQuery": ""} </code>	status: 200 OK Empty string search should return all standard data files
TC_PROCESS_DATA_API_001_21	BV-SQ-07	<code>{"searchQuery": "' OR 1=1--"} </code>	The system should prevent SQL injection attacks Do not return all data
TC_PROCESS_DATA_API_001_22	BV-SQ-08	<code>{"searchQuery": "<script>alert('XSS')</script>"} </code>	The system should prevent XSS attacks Correctly escaped search results
TC_PROCESS_DATA_API_001_23	BV-SQ-09	<code>{"searchQuery": "1; DROP TABLE users;"} </code>	The system should prevent destructive SQL injection and not execute malicious SQL commands

7.1.2 Simulation function (API for getting simulation results)

Parameter Table

parameter	Parameter description	Parameter required
standardDataId	Standard Data ID	Yes
deviceId	Device ID	Yes
file	Uploaded data file	Yes

A. Equivalence classes

modelId parameter

Type	Value	Description
Valid	1~N (existing ID)	Valid standard data ID existing in the database
Invalid	null, not provided	Standard data ID not provided
Invalid	Non-existent ID	Non-existent standard data ID

deviceId parameter

Type	Value	Description
Valid	1~N (existing ID)	Valid device ID existing in the database
Invalid	null, not provided	Device ID not provided
Invalid	Non-existent ID	Device ID does not exist

file parameter

Type	Value	Description
Valid	Non - empty CSV file (with only one row of data)	A valid CSV data file containing a header row and a row of data
Invalid	null, not provided	No file provided
Invalid	Non - CSV file	Non - CSV files such as.txt,.xlsx etc.
Invalid	CSV with non-compliant content	CSV file missing necessary data columns
Invalid	CSV with multiple rows of data	A CSV file containing more than one row of data
Invalid	Large files	CSV files larger than 16MB
Invalid	Empty file	Files with size 0 or only headers

B. Decision Table

Condition pile

- C1: Is modelId valid? (Valid/Invalid/Not provided)

- C2: Is deviceId valid? (Valid/Invalid/Not provided)
- C3: Is the file valid? (Valid/Not provided/Invalid format/Inconsistent content/Multiple lines of data/Too large/Empty)

Action Pile

- A1: Return 200 OK and simulation results
- A2: Return 400 Bad Request (parameter error)
- A3: Returns 413 Payload Too Large

Rule Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Conditions														
C1: standardDataId	Valid	Not Provided	Invalid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid
C2: deviceId	Valid	Valid	Valid	Not Provided	Invalid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid	Valid
C3: File	Valid Small	Valid Small	Valid Small	Valid Small	Valid Small	Valid Medium	Valid Large	Not Provided	Invalid Format (txt)	Invalid Format (xlsx)	Invalid Content (Missing Columns)	Invalid Content (Format Error)	Over sized	Empty (0 bytes)
Actions														
A1: 200 OK	✓					✓	✓							
A2: 400 Bad Request		✓	✓	✓	✓			✓	✓	✓	✓	✓		✓
A3: 413 Payload Too Large													✓	

C. Test case design

Get mock result API test case

Project	Content
Identification	TC_SIM_API_001
Test items	Basic functional testing for getting mock results API
Input specifications	POST /api/simulation/getSimulationResult Test multiple input parameter combinations
Output specifications	Return corresponding simulation results based on different input parameters
Environmental needs	API testing environments such as Mail carrier or other API testing tools
Special procedural requirements	A valid authentication token is required (if required). Need to prepare different types of test CSV files
Inter-case dependencies	None

Sub Use Case

Identification	Rule numb	Conditional combination	Input specifications	Output specifications
----------------	-----------	-------------------------	----------------------	-----------------------

	er			
TC_SIM_API_001_01	Rule 1	Valid standard data file ID + Valid deviceId + Valid CSV file (one line of data)	{"standardDataId": 1, "deviceId": 1, "file": [valid CSV file with one line of data]}	Status: 200 OK The response body contains successful simulation results, including image URLs, injury sites, remaining lifespan, and health indices
TC_SIM_API_001_02	Rule 2	No standard data file ID + valid deviceId + valid CSV file provided (one line of data)	{"deviceId": 1, "file": [valid CSV file with one line of data]}	Status: 400 Bad Request Response body contains error message for missing standardDataId
TC_SIM_API_001_03	Rule 3	Invalid standard data file ID + valid deviceId + valid CSV file (one line of data)	{"standardDataId": 999, "deviceId": 1, "file": [valid CSV file with one line of data]}	Status: 400 Bad Request The response body contains an error message that the standard data file does not exist
TC_SIM_API_001_04	Rule 4	Valid standard data file ID + no deviceId provided + valid CSV file (one line of data)	{"standardDataId": 1, "file": [valid CSV file with one line of data]}	Status: 400 Bad Request Response body contains error message for missing deviceId
TC_SIM_API_001_05	Rule 5	Valid standard data file ID + invalid deviceId + valid CSV file (one line of data)	{"standardDataId": 1, "deviceId": 999, "file": [valid CSV file with one line of data]}	Status: 400 Bad Request The response body contains an error message that the device does not exist
TC_SIM_API_001_06	Rule 6	Valid standard data file ID + Valid deviceId + No file provided	{"standardDataId": 1, "deviceId": 1}	Status: 400 Bad Request The response body contains an error message for missing files
TC_SIM_API_001_07	Rule 7	Valid standard data file ID + valid deviceId + invalid format file (txt)	{"standardDataId": 1, "deviceId": 1, "file": [.txt file]}	Status: 400 Bad Request The response body contains information about file format errors
TC_SIM_API_001_08	Rule 8	Valid standard data file ID + valid deviceId	{"standardDataId": 1, "deviceId": 1, "file":	Status: 400 Bad Request

		+ invalid format file (xlsx)	[.xlsx file]}	The response body contains information about file format errors
TC_SIM_API_001_09	Rule 9	Valid standard data file ID + valid deviceId + content does not match the file (missing column).	{"standardDataId": 1, "deviceId": 1, "file": [CSV missing required data columns]}	Status: 400 Bad Request The response body contains an error message for missing necessary data columns
TC_SIM_API_001_10	Rule 10	Valid standard data file ID + valid deviceId + content does not match the file (format error).	{"standardDataId": 1, "deviceId": 1, "file": [CSV with incorrect data format]}	Status: 400 Bad Request The response body contains information about data format errors
TC_SIM_API_001_11	Rule 11	Valid standard data file ID + Valid deviceId + CSV containing multiple rows of data	{"standardDataId": 1, "deviceId": 1, "file": [CSV containing multiple lines of data]}	Status: 400 Bad Request The response body contains an error message that only allows one line of data
TC_SIM_API_001_12	Rule 12	Valid standard data file ID + Valid deviceId + Large file	{"standardDataId": 1, "deviceId": 1, "file": [CSV file exceeding 16MB]}	Status: 413 Payload Too Large Response body contains error message of file too large
TC_SIM_API_001_13	Rule 13	Valid standard data file ID + valid deviceId + empty file (0 bytes)	{"standardDataId": 1, "deviceId": 1, "file": [0 byte file]}	Status: 400 Bad Request Response body contains an error message for an empty file
TC_SIM_API_001_14	Rule 14	Valid standard data file ID + Valid deviceId + Empty file (header only)	{"standardDataId": 1, "deviceId": 1, "file": [CSV file with headers only]}	Status: 400 Bad Request Response body contains error message of insufficient data

7.2 White-box test cases

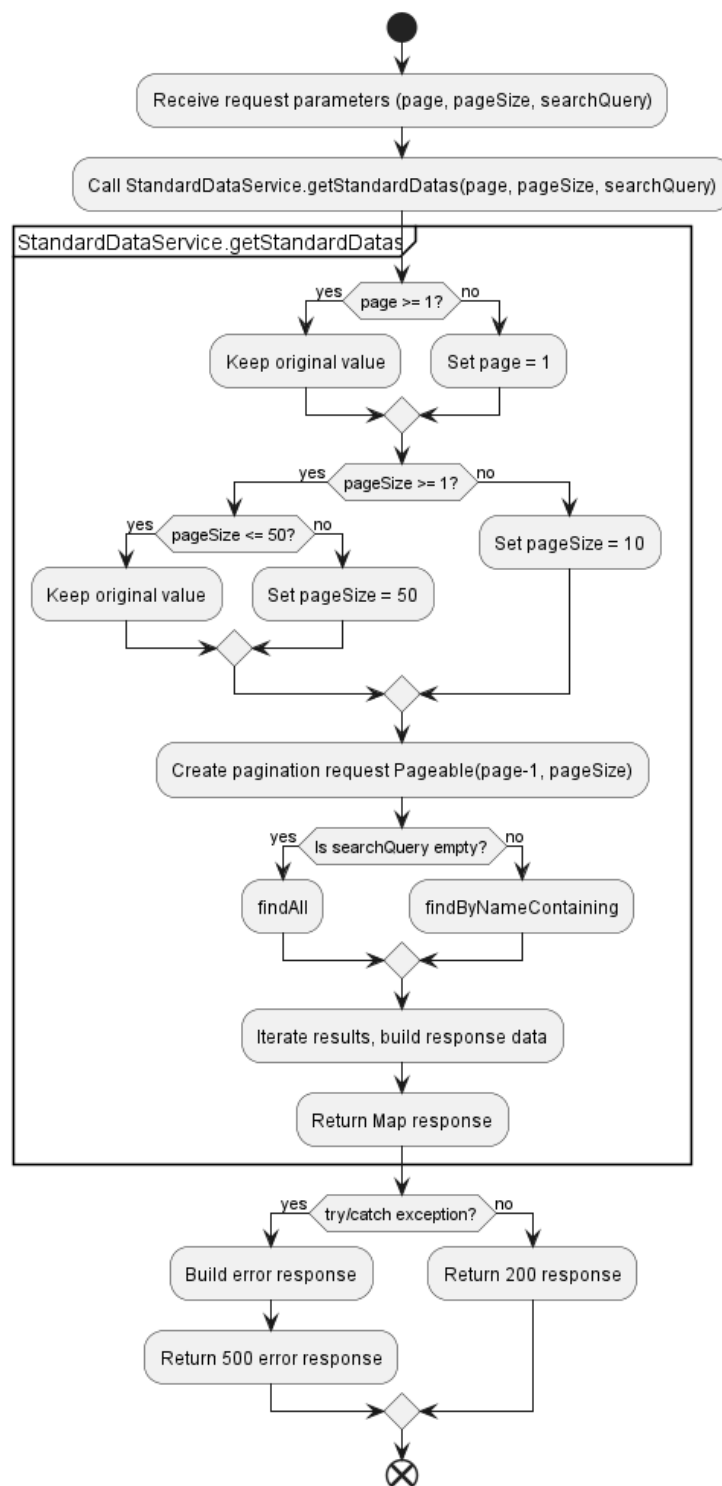
7.2.1 Get the standard data list function

Method Overview

Method name	Function	Layer

getStandardDatas	Get the standard data list, support paging and searching	Controller
getStandardDatas	Query the standard data list according to the paging parameters and search keywords	Service
findAll	Query all standard data (paging)	Repository
findByNameContaining	Query standard data by name keyword (paging)	Repository

Flowchart



7.2.1.1 Test case

A. Unit test

StandardDataController

Test Number	Test content	Input	Expected output	Target coverage point
UT_STANDARD_DATA_CTRL_001	Basic Function Test - Default Parameters	page=1, pageSize=10, searchQuery=null	Status code 200, success flag true, standard data list and total number	Normal call service path
UT_STANDARD_DATA_CTRL_002	Paging parameter test - valid parameter	page=2, pageSize=15, searchQuery=null	Status code 200, success flag true, standard data list and total number	Handling of different paging parameters
UT_STANDARD_DATA_CTRL_003	Search function test - with search terms	page=1, pageSize=10, searchQuery="turbine"	Status code 200, success flag true, search result list	Processing of Search Parameters
UT_STANDARD_DATA_CTRL_004	Abnormal Test - Service Abnormal	page=1, pageSize=10, searchQuery=null (Service throws exception)	Status code 500, success flag false, error message	Exception handling branch
UT_STANDARD_DATA_CTRL_005	Boundary test - zero or negative value parameter	page=0, pageSize=-5, searchQuery=null	Status code 200, service layer processing parameter correction	Abnormal parameter handling

StandardDataServiceImpl

Test Number	Test content	Input		
-------------	--------------	-------	--	--

			Expected output	Target coverage point
UT_STANDARD_DATA_SVC_001	Basic Function Test - Default Parameters	page=1, pageSize=10, searchQuery=null	Success flag true, standard data list and total number	Normal execution path
UT_STANDARD_DATA_SVC_002	Parameter correction test - page number is 0	page=0, pageSize=10, searchQuery=null	Success flag true, page number corrected to 1	page < 1 correction branch
UT_STANDARD_DATA_SVC_003	Parameter calibration test - pageSize is 0	page=1, pageSize=0, searchQuery=null	Success flag true, pageSize corrected to 10	pageSize < 1 correction branch
UT_STANDARD_DATA_SVC_004	Parameter correction test - pageSize exceeds the maximum value	page=1, pageSize=100, searchQuery=null	Success flag true, pageSize corrected to 50	pageSize > 50 correction branch
UT_STANDARD_DATA_SVC_005	Search function test - matching results	page=1, pageSize=10, searchQuery="turbine"	Success flag true, matching standard data list	searchQuery non - empty branch
UT_STANDARD_DATA_SVC_006	Search function test - blank search term	page=1, pageSize=10, searchQuery=" "	Success flag true, all standard data lists	searchQuery.trim() is empty branch
UT_STANDARD_DATA_SVC_007	Warehouse anomaly test	page=1, pageSize=10, searchQuery=null (Warehouse throws exception)	Throws RuntimeException	Exception handling

B. Integration Testing

StandardDataController + StandardDataServiceImpl

--	--	--	--	--

Test Number	Test content	Input	Expected output	Target coverage point
IT_STANDARD_DATA_CTRL_001	Basic Function Test - Default Parameters	page=1, pageSize=10, searchQuery=null	Status code 200, success flag true, standard data list and total number	Complete execution path
IT_STANDARD_DATA_CTRL_002	Paging parameter test - valid parameter	page=2, pageSize=15, searchQuery=null	Status code 200, success flag true, page 2 data	Paging function integrity
IT_STANDARD_DATA_CTRL_003	Parameter correction test - page number is 0	page=0, pageSize=10, searchQuery=null	Status code 200, page number corrected to 1	page < 1 correction branch
IT_STANDARD_DATA_CTRL_004	Parameter calibration test - pageSize is 0	page=1, pageSize=0, searchQuery=null	Status code 200, pageSize corrected to 10	pageSize < 1 correction branch
IT_STANDARD_DATA_CTRL_005	Parameter correction test - pageSize exceeds the maximum value	page=1, pageSize=100, searchQuery=null	Status code 200, pageSize corrected to 50	pageSize > 50 correction branch
IT_STANDARD_DATA_CTRL_006	Search function test - matching results	page=1, pageSize=10, searchQuery="turbine"	Status code 200, matching standard data list	Search function integrity
IT_STANDARD_DATA_CTRL_007	Search function test - no matching results	page=1, pageSize=10, searchQuery="Non-existent standard data"	Status code 200, empty list	Search no results handling
IT_STANDARD_DATA_CTRL_008	Search boundary test - empty search term	page=1, pageSize=10, searchQuery=""	Status code 200, all standard data lists	Empty search term handling
IT_STANDARD_DATA_CTRL_009	Abnormal test - Repository	page=1, pageSize=10,	Status code 500, error message	Exception handling branch

	exception	searchQuery=null (Warehouse throws exception)		
IT_STANDARD_DATA_CTRL_010	Combined Test - Multi-parameter Calibration	page=0, pageSize=100, searchQuery="Engine"	Status code 200, multiple parameters are corrected simultaneously	Parameter correction combination

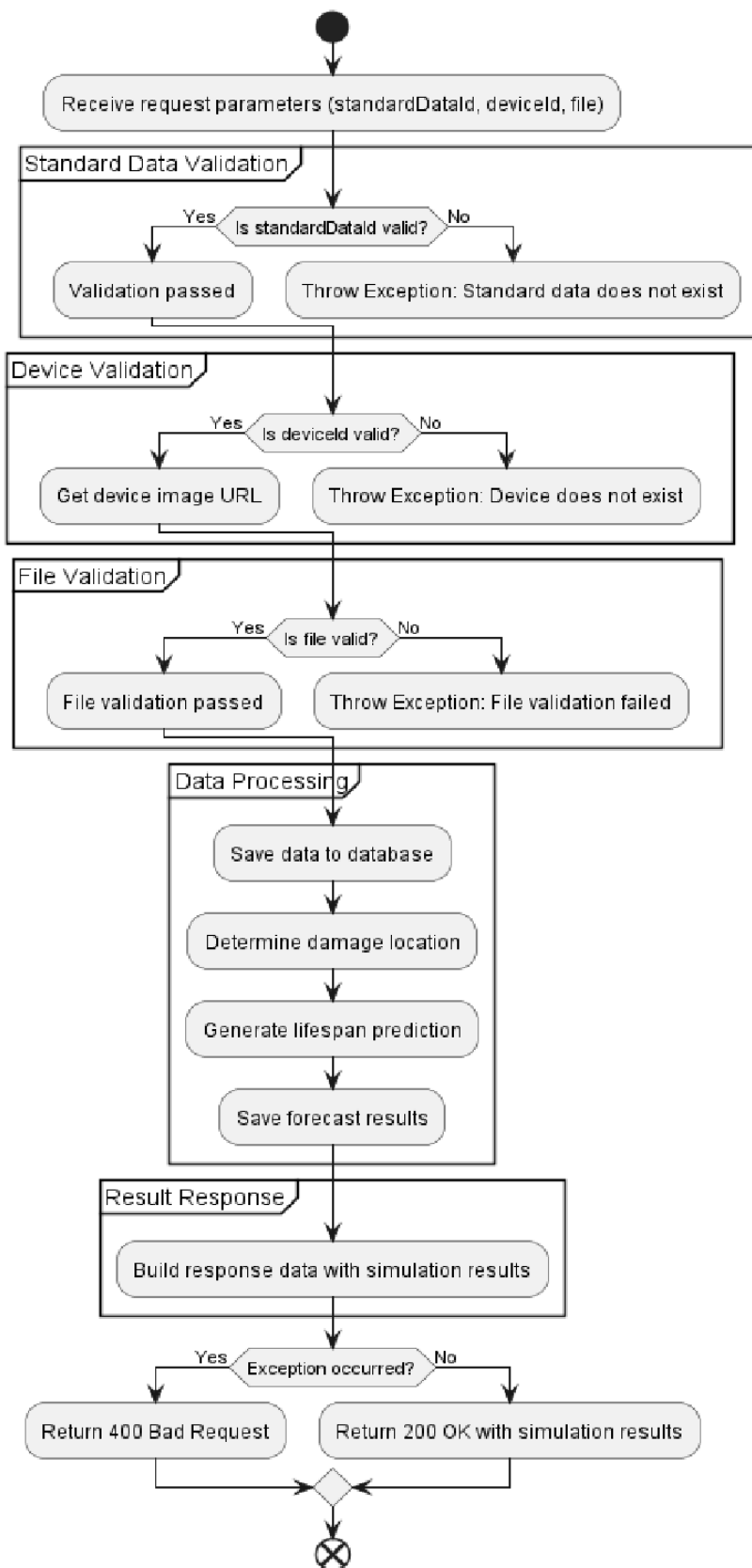
7.2.2 Get Simulation Results Function

Method Overview

Method name	Function	Layer
getSimulationResult	Process the simulation request and receive the standard data ID, device ID, and CSV file	Controller
getSimulationResult	Perform simulation calculations and return results	Service
validateModel	Verify that the standard data exists and is valid	Service
validateComponent	Verify that the device exists and is valid	Service
validateCsvFile	Verify the format and content of the CSV file	Service
saveUploadedFile	Save the uploaded CSV file	Service
saveDataFromFile	Save CSV data to database	Service
processData	Perform Data Analysis and Predictive Calculations	Service
saveForecastResult	Save prediction results to database	Service
buildResponse	Build API Response	Service
setDoubleValueIfExists	Set Attribute Values from CSV Data	Service
createDirectoryIfNotExists	Create the necessary directories	Service

Flowchart

Simplified Aircraft Engine Lifespan Simulation Process



7.2.2.1 Test case

A. Unit test

SimulationController

--	--	--	--	--

Test Number	Test content	Input	Expected output	Target coverage point
UT_SIM_CTRL_001	Basic Function Test - Normal Response	standardDataId = 1, deviceId = 1, valid file	Status code 200, success flag true, simulation result data	Normal call service path
UT_SIM_CTRL_002	Exception testing - service exception	standardDataId = 1, deviceId = 1, valid file (service throws exception)	Status code 400, success flag false, error message	Exception handling branch
UT_SIM_CTRL_003	Boundary test - file is empty	standardDataId = 1, deviceId = 1, empty file	Status code 400, success flag false, error message	File validation error handling

SimulationServiceImplementation

Test Number	Test content	Input	Expected output	Target coverage point
UT_SIM_SERV_001	Standard Data Validation Test - Invalid ID	standardDataId = 999, deviceId = 1, valid file	Throw exception: "Standard data does not exist"	Invalid standard data ID verification
UT_SIM_SERV_002	Standard data validation test - file does not exist	standardDataId = 2, deviceId = 1, valid file	Throw exception: "Standard data does not exist: [File path]"	Standard data file does not exist verification
UT_SIM_SERV_003	Device Verification Test - Invalid ID	standardDataId = 1, deviceId = 999, valid file	Throw exception: "Device does not exist"	Invalid device ID verification
UT_SIM_SERV_004	Equipment Picture Test - No Picture	standardDataId = 1, deviceId = 3, valid file	Return imageUrl as the default image URL.	Device no image URL processing
UT_SIM_SERV_005	Device picture test - picture does not exist	standardDataId = 1, deviceId = 4, valid file	Return imageUrl as the default image URL.	Device picture does not exist for processing
UT_SIM_SERV_006	File verification test - empty file	standardDataId = 1, deviceId = 1,0	Throw exception: "No file provided"	Empty file processing

		byte file	or file is empty"	
UT_SIM_SERV_007	File verification test - header only	standardDataId = 1, deviceId = 1, only CSV of the header	Throw exception: "File contains only header, no data rows"	Table Header File Handling Only
UT_SIM_SERV_008	File Verification Test - Non-CSV	StandardDataId = 1, deviceId = 1, non-CSV file	Throw exception: "Invalid file format. Only CSV files are supported"	File format verification
UT_SIM_SERV_009	File validation test - multi-line data	standardDataId = 1, deviceId = 1, CSV for multi-row data	Throw exception: "Invalid file format. Only one data row is allowed"	Multi-line data file validation
UT_SIM_SERV_010	File validation test - missing columns	StandardDataId = 1, deviceId = 1, CSV missing necessary columns	Throw an exception: "Missing required columns: [column name list]"	Column inspection
UT_SIM_SERV_011	Data validation test - incomplete data column	standardDataId = 1, deviceId = 1, CSV with incomplete data columns.	Throw exception: "Incomplete data row: fewer values than headers"	Data integrity check
UT_SIM_SERV_012	Data validation test - non-numerical data	setDoubleValueIfExists method, non-numerical input	Throw exception: "Invalid numeric format for column [column name]"	Non-numerical data processing
UT_SIM_SERV_013	Data validation test - column does not exist	setDoubleValueIfExists method, non-existent column	Throw exception: "Required column not found: [column name]"	Missing column processing
UT_SIM_SERV_014	Testing of Different Equipment Types -	processData method, including	Return damageLocation	Logic based on device name

	Engine	the device name of "engine"	is "High Pressure Turbine"	
UT_SIM_SERV_015	Testing of Different Equipment Types - Fan	processData method, device name containing "fan"	Return damageLocation as "Fan Blade"	Logic based on device name
UT_SIM_SERV_016	Exception test - file size exceeded limit	StandardDataId = 1, deviceId = 1, files larger than 16MB	Throw exception: "File is too large. Maximum size allowed is 16MB"	File size limit check
UT_SIM_SERV_017	Test save prediction results	SaveForecastResult method, valid input	Return the correctly set Forecast object	Prediction result saving logic
UT_SIM_SERV_018	Test build response	buildResponse method, valid input	Return a properly formatted response Map.	Response build logic
UT_SIM_SERV_019	Test save data file method	SaveDataFromFile method, valid input	Save the correctly set Data object	Data storage logic
UT_SIM_SERV_020	Full Process Testing - Success Scenarios	standardDataId = 1, deviceId = 5 (engine), valid file	Return complete and valid results	Complete execution path

B. Integration Testing

SimulationController + SimulationServiceImpl

Test Number	Test content	Input	Expected output	Target coverage point
EN_SIM_CTRL_001	Basic Function Test - Normal Input	standardDataId = 1, deviceId = 1, valid CSV file	Status code 200, success flag true, simulation result data	Complete execution path
EN_SIM_CTRL_002	Standard Data Validation Test - Invalid ID	standardDataId = 999, deviceId = 1, valid file	Status code 400, success flag false, error message	Invalid standard data ID processing

EN_SIM_CTRL_003	Device Verification Test - Invalid ID	standardDataId = 1, deviceId = 999, valid file	Status code 400, success flag false, error message	Invalid device ID handling
EN_SIM_CTRL_004	Equipment Picture Test - No Picture	standardDataId = 1, deviceId = 3, valid file	Status code 200, imageUrl is the default value	Device has no image processing
EN_SIM_CTRL_005	File verification test - empty file	standardDataId = 1, deviceId = 1, empty file	Status code 400, error message	Empty file error handling
EN_SIM_CTRL_006	File Verification Test - Non-CSV	StandardDataId = 1, deviceId = 1, non-CSV file	Status code 400, error message	File format error handling
EN_SIM_CTRL_007	Testing of Different Equipment Types - Engine	standardDataId = 1, deviceId = 5, valid file	Status code 200, damageLocation is "High Pressure Turbine"	Device type processing
EN_SIM_CTRL_008	Testing of Different Equipment Types - Fan	standardDataId = 1, deviceId = 6, valid file	Status code 200, damageLocation is "Fan Blade"	Device type processing

8. Test tool implementation

8.1 Explanation of the selected testing framework/tool

8.1.1 JUnit 5

Architecture and modules

JUnit 5 adopts a Modularization design, divided into three main modules: JUnit Platform, JUnit Jupiter, and JUnit Vintage (compatible with older versions). This design makes the framework more flexible and scalable, and can adapt to the needs of different projects.

Lifecycle management

JUnit 5 provides rich annotations for lifecycle management, including:

- @BeforeEach and @AfterEach: Used for preparation and cleanup before and after testing methods
- @BeforeAll and @AfterAll: used for initialization and finalization of the entire test class

This mechanism makes the control of the testing environment simple and precise, ensuring that testing is performed in a controlled environment.

Nested test

The nested test function implemented by `@Nested` annotation allows developers to organize test cases according to business logic or functional modules, forming a hierarchical test structure. This is particularly important for testing complex systems, as it can reflect the relationships between system components and keep the test structure consistent with the business domain model.

Parameterized Test

Parameterized testing is an important feature of JUnit 5. With the `@ParameterizedTest` annotation and various data source annotations (such as `@ValueSource`, `@CsvSource`, etc.), the same test method can be used to test multiple sets of inputs and expected outputs, greatly reducing redundant code and improving test efficiency and maintainability.

Assertion mechanism

JUnit 5 provides an enhanced assertion API, including the traditional `assertEquals` and the new `assertThrows`, `assertTimeout`, etc. These assertion methods support lambda expressions, making assertions more concise and error messages more informative.

8.1.2 Deep Dive into JaCoCo

Working principle

JaCoCo performs instrumentation at the bytecode level and collects coverage data during program execution. This non-intrusive design enables it to obtain accurate coverage metrics without modifying the source code, making it suitable for various Java projects.

Coverage metrics

JaCoCo provides multi-dimensional coverage metrics:

- Instruction coverage: reflects the execution of Java bytecode instructions
- Branch coverage: Indicates the execution ratio of conditional branches
- Line coverage: shows how many lines of source code have been executed
- Method coverage: Measure the proportion of methods called
- Class coverage: Measures the loading of classes

These metrics together form a comprehensive evaluation system for code quality.

CI/CD Integration

In a continuous integration environment, JaCoCo can be seamlessly integrated with build tools to form an automated quality monitoring system. By configuring the coverage threshold as a quality gate, ensure that the newly submitted code maintains a high level of test coverage and prevent quality degradation.

Report and Analysis

The HTML report generated by JaCoCo intuitively displays the coverage data, marking the executed and unexecuted code in different colors to help developers quickly identify the weak links in the test. At the same time, the XML and CSV reports are easy to integrate with other tools to support more complex Data Analysis and visualization requirements.

Microservice support

In the microservice architecture environment, JaCoCo can aggregate the coverage data of multiple service modules through special configuration, provide an overall view, help the team grasp the test quality at the system level, ensure that the key business logic is fully tested, and support the quality management of large-scale projects.

8.2 Test script development and execution

8.2.1 Black box testing

Black box testing is a software testing method that focuses on verifying the functional behavior of software without considering its internal implementation structure. In our aero-engine life prediction system, black box testing plays a crucial role in ensuring the correctness of the system's external functions.

8.2.1.1 Test scope

Our Black box testing covered the following key aspects of the system:

- **REST API Interface Function Verification:** Ensure that all externally exposed API interfaces work as expected, including standard data acquisition interfaces and simulation result calculation interfaces.
- **Business Scenario E2E Test:** Verify the complete user process, from selecting standard data to uploading user data and then to obtaining simulation results.
- **Input and output data format verification :** Test the system's ability to process the CSV file format and data column requirements, as well as the correctness of the output result format.

8.2.1.2 Test focus

During the test execution, we paid special attention to the following aspects:

- **Interface contract compliance** : Verify that the API implementation meets the predefined interface specifications, including parameter requirements, return formats, and status codes.
- **Error handling mechanism** : Check whether the system's response is reasonable and user - friendly when facing abnormal situations such as invalid input and non - existent resources.
- **Boundary condition handling** : Test whether the behavior of the system is stable and reliable when dealing with boundary conditions such as extreme values and special inputs.
- **Performance Benchmark** : Evaluate the response time of key APIs to ensure satisfactory performance under normal load.

8.2.1.3 Test Strategy

- **Equivalence Class Partitioning**

By dividing the input data into valid and invalid equivalence classes, we significantly reduce the number of test cases required while maintaining test coverage.

Example Application : When testing the standard data list API, we divide the page parameter into the following equivalence classes:

- Valid equivalence classes: page numbers within the normal range (1~N), no parameters passed (using default values)
- Invalid equivalence class: non - positive page number, page number exceeding the total number of pages, non - integer page number

This partitioning allows us to select representative values for testing instead of testing every possible input value.

- **Boundary value analysis**

Boundary value analysis complements equivalence partitioning by testing values near the boundaries of equivalence classes to find potential problems.

Example application : For the pageSize parameter (valid range 1-50), we designed the following boundary value tests:

- Minimum effective value (1) and maximum effective value (50)
- Values below the lower limit (0, -1)
- Values above the limit (51, 1000)
- Special type values (10.5, "abc")

Through these boundary tests, we found the system's weaknesses in handling non - integer page numbers and page sizes, which may not be found in regular tests.

- **Error guessing method**

Based on the tester's experience and understanding of the system, predict possible error conditions and design targeted tests.

Example application : For the search function, we tested the following inputs that might cause problems:

- SQL injection attempt ("' OR 1=1--")
- XSS attack string ("alert('XSS')")
- Special character search ("Special characters! @ # \$%")

These tests helped us verify the system's protection against potential security threats.

- **Decision Table Method**

Test the system behavior for multiple input condition combinations to ensure that all logical branches are covered.

Example application: When testing the simulation function API, we built a decision table with the following conditions:

- Is the standard data ID valid (valid/invalid/not provided)?
- Whether the device ID is valid (valid/invalid/not provided)
- Is the file valid (valid/not provided/invalid format/content mismatch/multiple rows of data/too large/empty)?

Based on these condition combinations, we identified 14 different test rules that covered the system responses to various input combinations and effectively detected the inconsistent status code issue in file size validation.

8.2.1.4 Execution steps

Script Writing

- **Test script organization structure**

代码块

```
1  @SpringBootTest
2  @AutoConfigureMockMvc
3  @TestClassOrder(ClassOrderer.OrderAnnotation.class)
4  public class StandardDataApiBlackBoxTests {
5      // 测试类的主体内容
6
7      @Nested
8      @Order(1)
9      @DisplayName("A. 页码参数测试")
10     @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
```

```
11     class PaginationTests {
12         // 嵌套测试类的内容
13     }
14
15     // 其他嵌套测试类...
16 }
```

With `@TestClassOrder` and `@Order` annotations, we ensure that test classes and test methods are executed in a predefined order, which is important when testing interdependent functionality.

- **Standard data API test execution process**

- **Page number parameter test:**

- Perform the default paging test to verify that the system uses the default page number and page size when no parameters are provided
 - Test custom paging parameters to verify that the system can correctly handle different valid paging configurations
 - Perform boundary value testing, including zero page numbers, negative page numbers, and page numbers that exceed the total number of pages
 - Verify the system's handling mechanism for illegal page numbers (decimals, non-numeric strings)

- **Page size parameter test:**

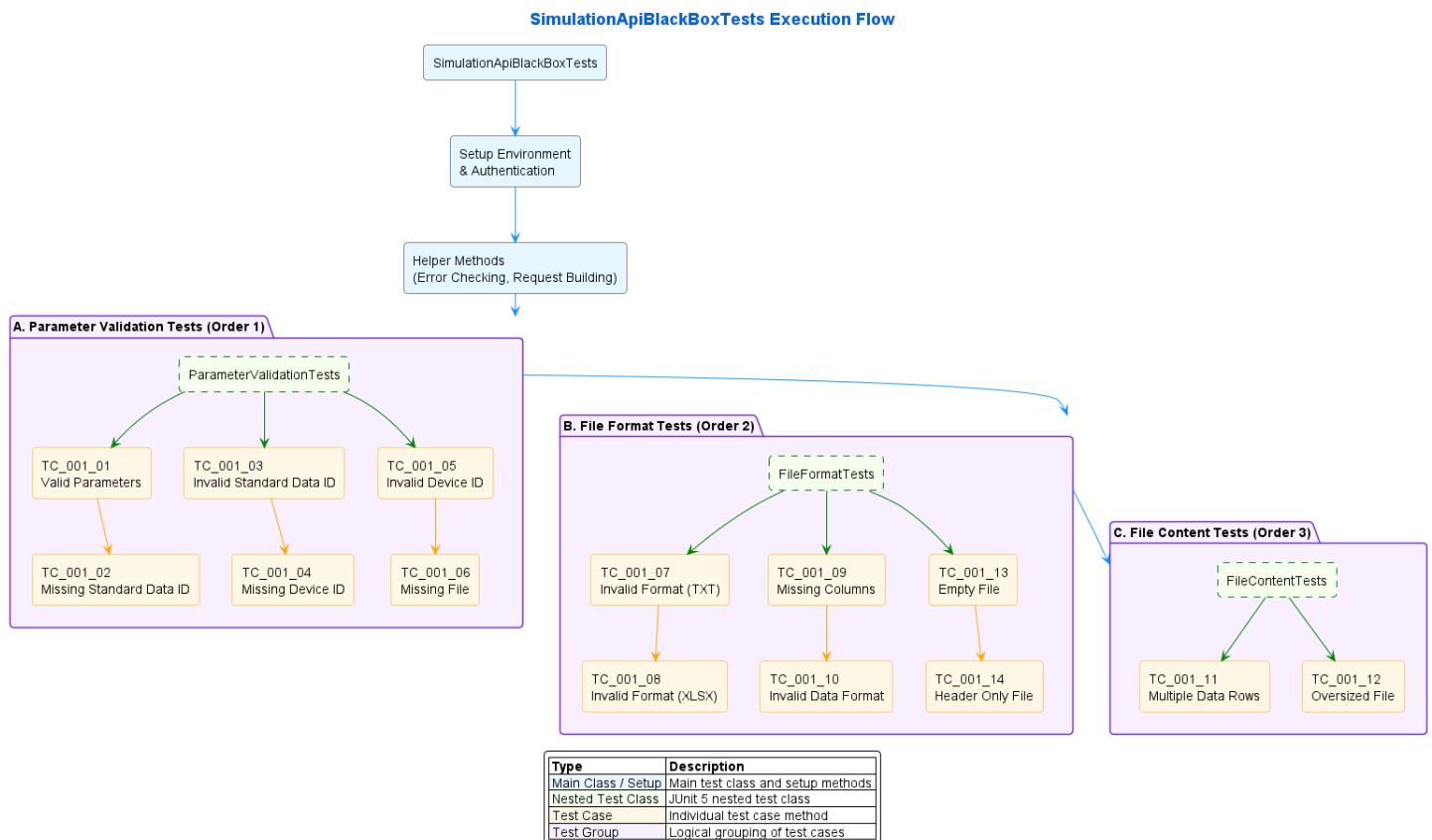
- Test zero or negative page size to verify the system's automatic correction mechanism
 - Verify that page sizes exceeding the maximum allowed value are properly restricted
 - Perform illegal page size input (decimal, non - numeric string) test and check system response

- **Search parameter test :**

- Test English keyword search to verify the system's support for Latin character search
 - Perform a Chinese keyword search to confirm the system's support for multilingual search
 - Test long string and special character search, verify the system's ability to handle abnormal input
 - Verify empty string search and confirm its consistency with the result of not providing search parameters

- **Security test:**

- Perform SQL injection tests to verify the system's ability to defend against malicious search parameters
- Test XSS attack scripts to confirm that the system correctly handles and escapes malicious code
- Verify the defense against destructive SQL injection to ensure that the system security is not affected

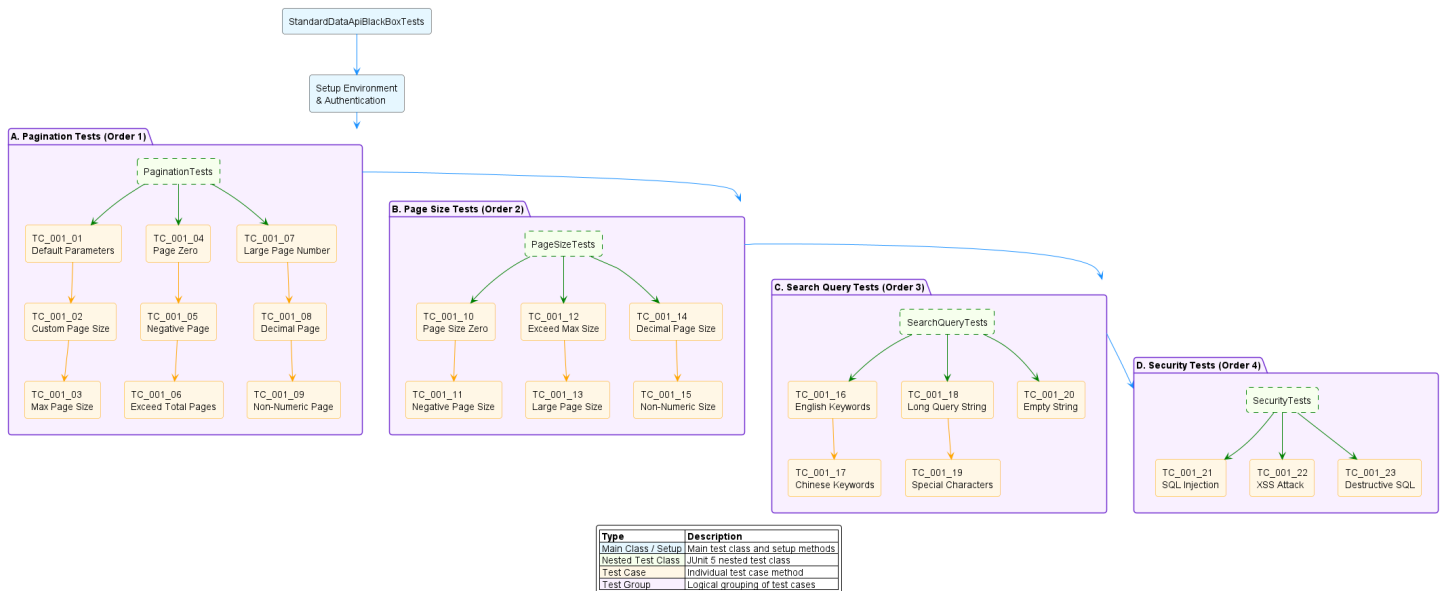


• API test execution process for simulation results

- **Environment preparation and tool method setting:**
 - Configure the authentication token acquisition mechanism to ensure API access rights
 - Implement a general error checking method to simplify the response verification logic
 - Create a file construction auxiliary method to support the generation of different types of test files
- **Parameter verification test :**
 - Test all parameters in a standard scenario to verify the normal function of the system
 - Perform missing standard data ID test to verify the necessary parameter check mechanism
 - Test invalid standard data ID to confirm the system's handling of non-existent resources

- Verify the error handling for missing device ID and invalid device ID
- Test the case where the file parameter is missing and confirm the parameter integrity verification
- **File format test :**
 - Test the upload of non-CSV format files (TXT, XLSX) and verify the format verification mechanism
 - Perform CSV structure error testing, including missing required columns and data format errors
 - Verify the processing of empty files and files containing only headers, and confirm the content integrity check
 - Test the processing logic and error messages for various boundary conditions
- **File content test:**
 - Test the multi-line data CSV file to verify the system's check on the data volume limit
 - Perform a super large file upload test to confirm the system's file size limit mechanism

StandardDataApiBlackBoxTests Execution Flow



Running method

1. Perform all Black box tests:

代码块

```
1 mvn test -Dtest="*BlackBoxTests"
```

2. Execute the specified test class:

```
代码块 mvn test -Dtest="StandardDataApiBlackBoxTests"
```

4. Generate test report:

代码块

```
1 mvn surefire-report:report
```

8.2.2 White-box testing

White - box testing is a software testing method that verifies the correctness of a system by examining its internal structure, code, and algorithms. In the aero - engine life prediction system, white - box testing allows us to understand and verify the reliability and integrity of the system's internal logic.

8.2.2.1 Test scope

Our white-box testing covers the following core levels of the system:

- **Business logic unit test** : Test the independent functional units of the service layer and tool classes to ensure that each component correctly implements its business logic. For example, we tested the paging logic in the standard data service and the data processing algorithm in the simulation calculation service.
- **Component integration test**: Verify the collaboration between multiple components, especially the interaction between the controller and the service layer. These tests ensure that data is correctly transferred and processed between system layers.
- **Exception handling path testing**: Specifically tests the system's error handling mechanism to verify that the system can respond gracefully and provide appropriate error messages under various abnormal conditions. For example, test abnormal paths such as the absence of standard data and incorrect file formats.

8.2.2.2 Test focus

During the white-box testing execution, we focus on the following aspects:

- **Internal logical branches of the method**: Verify the correctness of control flows such as conditional judgment, loop structure, and exception handling. For example, we focused on testing the judgment logic for different device types in the simulation service and multiple conditional branches in file verification.
- **Data flow and control flow** : Track the transfer path of data in the system to ensure that data flows correctly between different methods and components. Pay special attention to how CSV data is parsed, verified, and converted into model objects.

- **Boundary condition handling** : Test the behavior of the system when dealing with extreme values and special inputs, including null values, invalid IDs, and data with incorrect formats. For example, verify the automatic correction logic when the pagination parameter is zero or a negative value.

8.2.2.3 Test Strategy

- **Statement coverage**

Ensure that the test cases execute each statement in the program at least once to meet the basic code coverage requirements.

Example Application: For the standard data service, we designed test cases to ensure that all lines of code in each method are executed, including steps such as parameter validation, data query, and result construction.

- **Branch coverage**

The test case covers the true and false branches of each judgment condition in the program to ensure that all possible execution paths are tested.

Example Application: In the file validation logic of the mock service, we designed multiple test cases to cover all conditional branches

```
// 读取表头行
String headerLine = reader.readLine();
if (headerLine == null) {
    throw new Exception("File is empty");
}
```

```
if (values.length < headers.length) {
    throw new Exception("Incomplete data row: fewer values than headers");
}
```

- **Path coverage**

Designing test cases to cover all possible execution paths in the program is the strictest code coverage standard.

Example application : For the logic of processing device images, we designed test cases to cover all possible path combinations:

代码块

- 1 Analysis of device image processing paths:
- 2 - Path 1: The device exists and the image URL is valid (normal path).
- 3 - Path 2: The device exists but there is no image URL (TC_SIM_SERV_004).
- 4 - Path 3: The device exists but the image file does not exist (TC_SIM_SERV_005).
- 5 - Path 4: The device does not exist (TC_SIM_SERV_003).

Path coverage testing helped us identify processing defects in multiple extreme cases, especially error handling logic when resources do not exist.

8.2.2.4 Execution steps

Script Writing

- **Test preparation phase**
 - **Testing environment configuration :**
 - Configure the test database environment to ensure that test data is isolated from the production environment
 - Set up the test directory structure to store temporary files and test resources
 - Initialization testing framework (JUnit 5) and mock tool (Mockito)
 - **Test data preparation :**
 - Create simulation data, including standard data models, equipment components, test files, etc
 - Configure mock object behavior, set expected return values and exceptions
 - Prepare physical test files such as CSV data files and model files for integration testing
- **Unit test execution phase**
 - **Controller unit test :**
 - By simulating the service layer, test the controller's handling of request parameters
 - Verify that the controller encapsulates the results returned by the service layer correctly
 - Test the exception handling mechanism of the controller layer
 - **Service layer unit test :**
 - Simulate the data access layer to verify the correctness of business logic
 - Processing flow for testing complex condition branches
 - Verify the transformation and processing of data by the service layer
 - Test behavior under abnormal scenarios
- **Integration test execution phase**
 - **Controller and service layer integration testing :**
 - Test the interaction between the controller and the real service layer
 - Verify complete request processing flow
 - Test consistency between parameter validation and business rules
 - **Service layer and warehouse layer integration testing :**
 - Verify the data operation logic of the service layer using the simulated warehouse layer

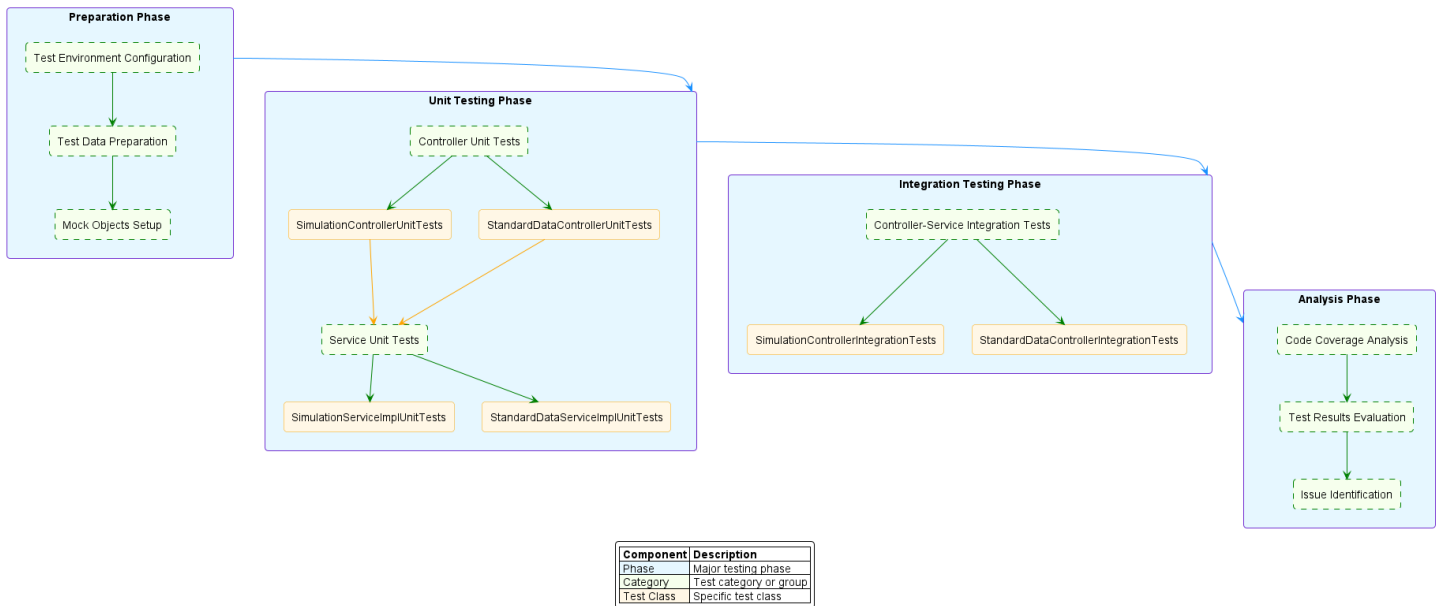
- Testing complex query and data processing flows
- Verify transaction processing and data consistency

• Test result analysis phase

◦ Coverage analysis :

- Generate code coverage report using JaCoCo tool
- Analyze instruction coverage, branch coverage, line coverage and other metrics
- Identify areas with insufficient test coverage

White Box Testing Process Flow



Running method

1. Execute the specified test class:

代码块

```
1 mvn test -Dtest=SimulationControllerUnitTests
```

2. Generate coverage report:

代码块

```
1 mvn jacoco:report
```

5. View test report:

- Unit test reports: `target/surefire-reports`
- Coverage report: `target/site/jacoco/index.html`

9. Analysis of test results

9.1 Black box testing

9.1.1 Results display

9.1.1.1 Select the standard data function (get the standard data list API)

Overall results

 测试失败: 4, 已通过: 19共 23 个测试 – 17秒 51毫秒

A. 页码参数测试5秒 876毫秒

TC_PROCESS_DATA_API_001_01: 默认分页参数1秒 484毫秒

TC_PROCESS_DATA_API_001_02: 自定义分页 - page=1, pageSize=1546毫秒

TC_PROCESS_DATA_API_001_03: 自定义分页 - page=2, pageSize=50543毫秒

TC_PROCESS_DATA_API_001_04: 边界值 - page=0605毫秒

TC_PROCESS_DATA_API_001_05: 边界值 - page=-1638毫秒

TC_PROCESS_DATA_API_001_06: 边界值 - 超出总页数994毫秒

TC_PROCESS_DATA_API_001_07: 边界值 - 极大页码646毫秒

TC_PROCESS_DATA_API_001_08: 边界值 - 小数页码202毫秒

TC_PROCESS_DATA_API_001_09: 边界值 - 非数字页码218毫秒

B. 分页大小参数测试2秒 592毫秒

TC_PROCESS_DATA_API_001_10: 边界值 - pageSize=0557毫秒

TC_PROCESS_DATA_API_001_11: 边界值 - pageSize=-1442毫秒

TC_PROCESS_DATA_API_001_12: 边界值 - pageSize=51417毫秒

TC_PROCESS_DATA_API_001_13: 边界值 - pageSize=1000870毫秒

TC_PROCESS_DATA_API_001_14: 边界值 - 小数页大小152毫秒

TC_PROCESS_DATA_API_001_15: 边界值 - 非数字页大小154毫秒

C. 搜索参数测试1秒 644毫秒

TC_PROCESS_DATA_API_001_16: 搜索 - 英文关键词218毫秒

TC_PROCESS_DATA_API_001_17: 搜索 - 中文关键词204毫秒

TC_PROCESS_DATA_API_001_18: 搜索 - 长字符串214毫秒

TC_PROCESS_DATA_API_001_19: 搜索 - 特殊字符201毫秒

TC_PROCESS_DATA_API_001_20: 搜索 - 空字符串807毫秒

D. 安全性测试1秒 309毫秒

TC_PROCESS_DATA_API_001_21: 安全 - SQL注入547毫秒

TC_PROCESS_DATA_API_001_22: 安全 - XSS攻击199毫秒

TC_PROCESS_DATA_API_001_23: 安全 - 破坏性SQL注563毫秒

Identification	Output specifications	Test result
TC_PROCESS_DATA_API_001_01	status: 200 OK The default values should be used: page=1, pageSize=10 Returns the standard data file data on page 1, with 10 items per page	PASS
TC_PROCESS_DATA_API_001_02	status: 200 OK Return data on page 1, 1 record per page	PASS
TC_PROCESS_DATA_API_001_03	status: 200 OK Return data on page 5, 50 records per page	PASS
TC_PROCESS_DATA_API_001_04	Should return an error response or automatically correct to page = 1	PASS
TC_PROCESS_DATA_API_001_05	Should return an error response or automatically correct to page = 1	PASS

TC_PROCESS_DATA_API_001_06	If the total number of pages is 5, an empty list or error message should be returned	PASS
TC_PROCESS_DATA_API_001_07	should return an empty list or an appropriate error message	PASS
TC_PROCESS_DATA_API_001_08	Should return an error response or automatically correct to a valid integer value	FAIL
TC_PROCESS_DATA_API_001_09	Should return an error response or automatically correct to the default value	FAIL
TC_PROCESS_DATA_API_001_10	Should return an error response or automatically correct to the default value of 10	PASS
TC_PROCESS_DATA_API_001_11	Should return an error response or automatically correct to the default value of 10	PASS
TC_PROCESS_DATA_API_001_12	Should return an error response or automatically correct to the maximum allowed value of 50	PASS
TC_PROCESS_DATA_API_001_13	Should return an error response or automatically correct to the maximum allowed value of 50	PASS
TC_PROCESS_DATA_API_001_14	Should return an error response or automatically correct to a valid integer value	FAIL
TC_PROCESS_DATA_API_001_15	Should return an error response or automatically correct to the default value of 10	FAIL
TC_PROCESS_DATA_API_001_16	status: 200 OK Returns a list of standard data files whose names contain the string "fan".	PASS
TC_PROCESS_DATA_API_001_17	status: 200 OK Returns a list of standard data files whose names contain the string "turbine".	PASS
TC_PROCESS_DATA_API_001_18	status: 200 OK The system should handle long search strings correctly	PASS

TC_PROCESS_DATA_API_001_19	status: 200 OK The system should handle special characters correctly	PASS
TC_PROCESS_DATA_API_001_20	status: 200 OK Searching for an empty string should return all standard data files	PASS
TC_PROCESS_DATA_API_001_21	The system should prevent SQL injection attacks Do not return all data	PASS
TC_PROCESS_DATA_API_001_22	The system should prevent XSS attacks Search results after correct escape	PASS
TC_PROCESS_DATA_API_001_23	The system should prevent destructive SQL injection Do not execute malicious SQL commands	PASS

Detailed description of wrong cases

- TC_PROCESS_DATA_API_001_08: Boundary value - Decimal page number**

```
MockHttpServletResponse:
    Status = 400
    Error message = null
    Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", X-Content-Type-Options:"nosniff", X-XSS-Protection:"1; mode=block"]
    Content type = null
    Body =
    Forwarded URL = null
    Redirected URL = null
    Cookies = []

java.lang.AssertionError: Status expected:<200> but was:<400>
预期:200
实际:400
<点击以查看差异>
```

- TC_PROCESS_DATA_API_001_09: Boundary value - Non-numeric page number**

```
MockHttpServletResponse:
    Status = 400
    Error message = null
    Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", X-Content-Type-Options:"nosniff", X-XSS-Protection:"1; mode=block"]
    Content type = null
    Body =
    Forwarded URL = null
    Redirected URL = null
    Cookies = []

java.lang.AssertionError: Status expected:<200> but was:<400>
预期:200
实际:400
<点击以查看差异>
```

- TC_PROCESS_DATA_API_001_14: Boundary value - Decimal page size**

```
MockHttpServletResponse:
    Status = 400
    Error message = null
    Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", X-Content-Type
    Content type = null
    Body =
    Forwarded URL = null
    Redirected URL = null
    Cookies = []

java.lang.AssertionError: Status expected:<200> but was:<400>
预期:200
实际:400
<点击以查看差异>
```

- **TC_PROCESS_DATA_API_001_15: Boundary value - Non-numeric page size**

```
MockHttpServletResponse:
    Status = 400
    Error message = null
    Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", X-Content-Type-Options:"n
    Content type = null
    Body =
    Forwarded URL = null
    Redirected URL = null
    Cookies = []

java.lang.AssertionError: Status expected:<200> but was:<400>
预期:200
实际:400
<点击以查看差异>
```

Cause Analysis

- **Parameter type mismatch:**
 - Error message: `MethodArgumentTypeMismatchException: Method parameter 'page': Failed to convert value of type 'java.lang.String' to required type 'int'`
 - The backend controller defines the parameters `page` and `pageSize` as `int` type
 - When a non - integer value (such as "1.5", "abc") is passed in, Spring MVC cannot convert it to an integer type
- **Expectations do not match actual behavior :**
 - Expected status code 200 OK, but got 400 Bad Request
 - The test expects the system to prompt for invalid data, but the system actually returns an error
- **Error handling method :**
 - The current API implementation uses strict type checking rather than fault tolerance

- For invalid input, the system should attempt to correct and return information that is easy for the user to view

Risk Warning

- User Experience Risk:**
 - When the user enters an illegal value, a technical error (status code 400) is received without a friendly prompt
 - May cause the front - end application to crash or display a generic error message, leaving the user unable to understand the problem
- Interface consistency risk :**
 - The API documentation may not match the actual behavior, causing confusion for integrators
 - Integrated clients may fail if they do not handle these error conditions correctly
- Test coverage risk :**
 - The current test expectations do not match the actual behavior of the API, which may result in other edge cases not being properly tested
- System robustness risk :**
 - Strict type checking improves security but reduces the system's fault tolerance for abnormal inputs

9.1.1.2 Simulation function (API for obtaining simulation results)

Overall results



A. 参数验证测试		4秒 943毫秒
✓ TC_SIM_API_001_01: 有效标准数据文件ID + 有效deviceId + 有效CSV文件(一行数据)		2秒 964毫秒
✓ TC_SIM_API_001_02: 未提供标准数据文件ID + 有效deviceId + 有效CSV文件(一行数据)		197毫秒
✓ TC_SIM_API_001_03: 无效标准数据文件ID + 有效deviceId + 有效CSV文件(一行数据)		528毫秒
✓ TC_SIM_API_001_04: 有效标准数据文件ID + 未提供deviceId + 有效CSV文件(一行数据)		213毫秒
✓ TC_SIM_API_001_05: 有效标准数据文件ID + 无效deviceId + 有效CSV文件(一行数据)		836毫秒
✓ TC_SIM_API_001_06: 有效标准数据文件ID + 有效deviceId + 未提供文件		185毫秒

B. 文件格式测试		5秒 141毫秒
✓ TC_SIM_API_001_07: 有效标准数据文件ID + 有效deviceId + 无效格式文件(txt)		916毫秒
✓ TC_SIM_API_001_08: 有效标准数据文件ID + 有效deviceId + 无效格式文件(xlsx)		845毫秒
✓ TC_SIM_API_001_09: 有效标准数据文件ID + 有效deviceId + 内容不符文件(缺列)		803毫秒
✓ TC_SIM_API_001_10: 有效标准数据文件ID + 有效deviceId + 内容不符文件(格式错)		895毫秒
✓ TC_SIM_API_001_13: 有效标准数据文件ID + 有效deviceId + 空文件(0字节)		793毫秒
✓ TC_SIM_API_001_14: 有效标准数据文件ID + 有效deviceId + 空文件(只有表头)		889毫秒

C. 文件内容测试		1秒 831毫秒
✓ TC_SIM_API_001_11: 有效标准数据文件ID + 有效deviceId + 包含多行数据的CSV		841毫秒
✗ TC_SIM_API_001_12: 有效标准数据文件ID + 有效deviceId + 超大文件		990毫秒

Identification	Output specifications	Test result
TC_SIM_API_001_01	status: 200 OK	PASS

	The response body contains the successful simulation results, including the image URL, damage location, remaining life, and health index	
TC_SIM_API_001_02	Status: 400 Bad Request Response body contains error message for missing standardDataId	PASS
TC_SIM_API_001_03	Status: 400 Bad Request The response body contains an error message that the standard data file does not exist	PASS
TC_SIM_API_001_04	Status: 400 Bad Request Response body contains error message for missing deviceId	PASS
TC_SIM_API_001_05	Status: 400 Bad Request The response body contains an error message that the device does not exist	PASS
TC_SIM_API_001_06	Status: 400 Bad Request The response body contains an error message for missing files	PASS
TC_SIM_API_001_07	Status: 400 Bad Request The response body contains information about file format errors	PASS
TC_SIM_API_001_08	Status: 400 Bad Request The response body contains information about file format errors	PASS
TC_SIM_API_001_09	Status: 400 Bad Request The response body contains an error message for missing necessary data columns	PASS
TC_SIM_API_001_10	Status: 400 Bad Request The response body contains information about data format errors	PASS
TC_SIM_API_001_11	Status: 400 Bad Request The response body contains an error message that only allows one line of data	PASS
TC_SIM_API_001_12	Status: 413 Payload Too Large	FAIL

	The response body contains an error message that the file is too large	
TC_SIM_API_001_13	Status: 400 Bad Request Response body contains an error message for an empty file	PASS
TC_SIM_API_001_14	Status: 400 Bad Request Response body contains error message of insufficient data	PASS

Detailed description of wrong cases

- TC_SIM_API_001_12: Valid standard data file ID + valid deviceId + oversized file

```
MockHttpServletResponse:
    Status = 400
    Error message = null
    Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers",
    Content type = application/json
    Body = {"success":false,"message":"File is too large. Maximum size allowed is 16MB"}
    Forwarded URL = null
    Redirected URL = null
    Cookies = []

java.lang.AssertionError: Status expected:<413> but was:<400>
预期:413
实际:400
<点击以查看差异>
```

Cause Analysis

- **Implementation does not match specification:**
 - Error message: Status expected:<413> but was:<400>
 - The test case expected the API to return an HTTP status code of 413 (Payload Too Large) for an oversized file, but the API returned 400 (Bad Request) instead
 - The test cases are designed based on REST API best practices, where an oversized payload should return 413, but the actual implementation uses 400 for all validation errors
- **Inconsistent error handling methods:**
 - Threw a generic Exception with error message "File is too large. Maximum size allowed is 16MB"
 - Catch all exceptions and return a 400 Bad Request status without distinguishing the type of error

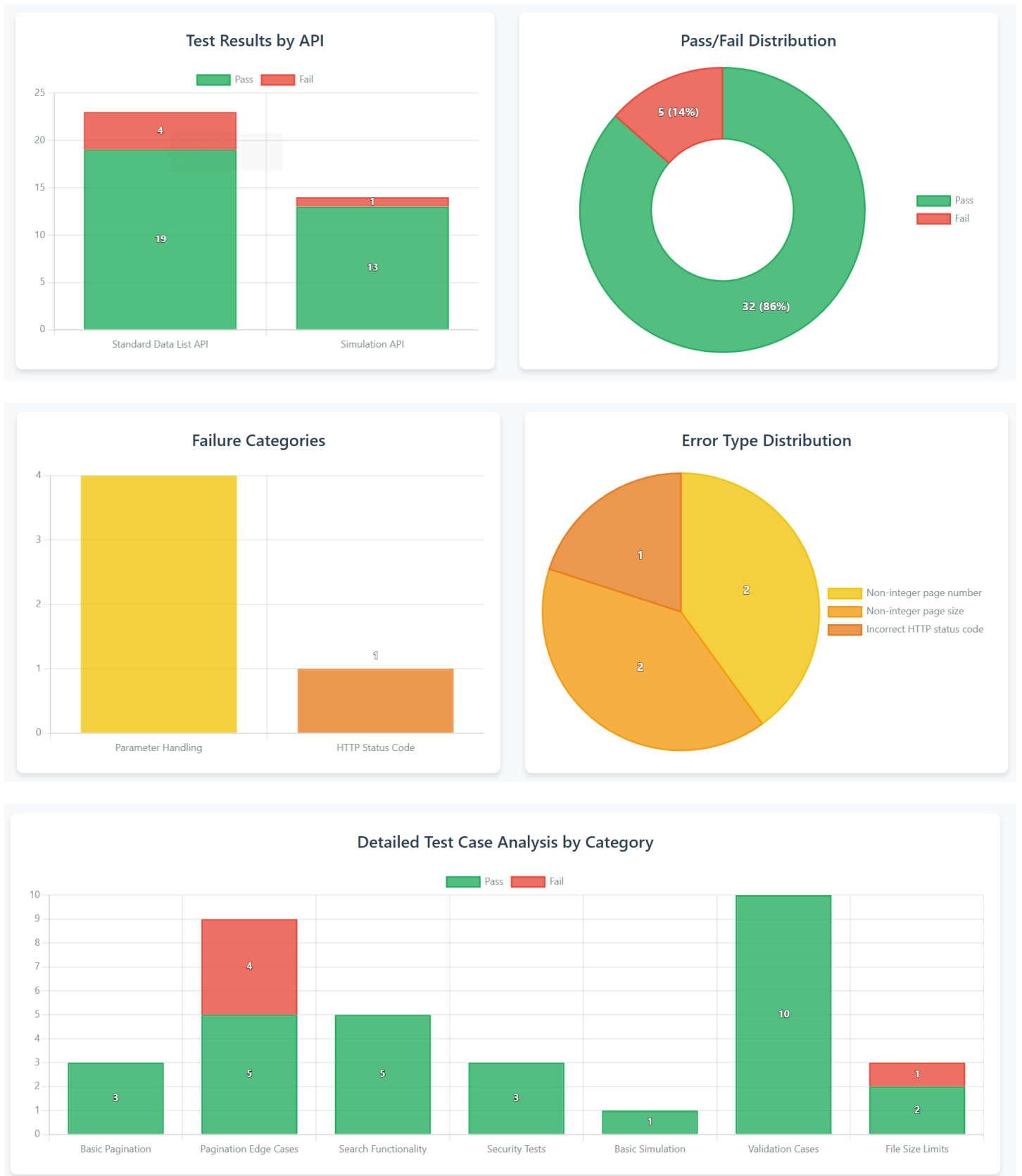
- There is no specific handling logic for different error categories (e.g., file size error vs parameter error)
- **Database transaction analysis:**
 - Hibernate logs show successful execution of database queries for user authentication and model/component validation
 - The error occurred before any mock data was stored, indicating that the size validation occurred early in the process
 - When the file size verification fails, the transaction is not committed properly

Risk Warning

- **API specification compliance risk:**
 - API behavior does not follow HTTP protocol conventions, where status code 413 is specifically designed for payload size issues
 - Integration partners expecting standard HTTP status codes may misinterpret these responses
 - It can be misleading if the API documentation specifies standard HTTP semantics that are not followed
- **Client integration risk:**
 - The Client may implement retry logic based on HTTP status codes and may inappropriately retry uploading an oversized file when receiving 400 instead of 413
 - The front - end application may not display the correct error message to the user, causing the user to be confused about the reason for the file upload failure
- **Scalability and resource management risks:**
 - Without an appropriate 413 response, the Client may not understand the system's file size limits
 - This may lead to repeated attempts to upload very large files, unnecessarily consuming internet bandwidth and server resources
 - The current implementation still checks the file size before processing, but the incorrect status code weakens the protection effect
- **Maintenance and debugging risks:**
 - Generic error handling makes it difficult to distinguish between different types of errors
 - When debugging complex problems, non - specific error status codes may increase the difficulty of troubleshooting

- Future developers may have difficulty understanding the current error handling logic, increasing maintenance costs

9.1.2 Statistical Chart



9.2 White-box testing results

9.2.1 Select the standard data function (get the standard data list API)

9.2.1.1 Unit test

StandardDataController

- Result screenshot

✓ StandardDataControllerUnitTests (com.example.software_management.Controller)

1秒 432毫秒

✓ UT_STANDARD_DATA_CTRL_001: 基本功能测试-默认参数

1秒 418毫秒

✓ UT_STANDARD_DATA_CTRL_002: 分页参数测试-有效参数

3毫秒

✓ UT_STANDARD_DATA_CTRL_003: 搜索功能测试-有搜索词

2毫秒

✓ UT_STANDARD_DATA_CTRL_004: 异常测试-Service异常

7毫秒

✓ UT_STANDARD_DATA_CTRL_005: 边界测试-零或负值参数

2毫秒

```
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.software_management.Controller.SimulationControllerUnitTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.695 s - in com.example.software_management.Controller.SimulationControllerUnitTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.11:report (default-report) @ Backend ---
[INFO] Loading execution data file F:\AAAFourthGrade\SoftwareManagement\TJ-Software-management\Codes\Backend\target\jacoco.exec
[INFO] Analyzed bundle 'Software_Management' with 49 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 20.142 s
[INFO] Finished at: 2025-06-09T00:34:54+08:00
[INFO] -----
```

- Results Table

Test number	Test content	Target coverage point	Test result
UT_STANDARD_DATA_CTRL_001	Basic function test - default parameters	Normal call service path	PASS
UT_STANDARD_DATA_CTRL_002	Paging parameter test - valid parameter	Handling of different paging parameters	PASS
UT_STANDARD_DATA_CTRL_003	Search function test - with search terms	Handling of search parameters	PASS
UT_STANDARD_DATA_CTRL_004	Abnormal test - Service exception	Exception handling branch	PASS
UT_STANDARD_DATA_CTRL_005	Boundary test - zero or negative parameter	Abnormal parameter handling	PASS

- **Test coverage description**
 - The instruction coverage is 100%, covering all 42 instructions
 - All methods (2 in total) have been fully tested
 - The line coverage is good, covering a total of 11 lines of code
 - No branch coverage metrics indicate that the controller logic is relatively straightforward

Software_Management > com.example.software_management.Controller > StandardDataController

StandardDataController

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
getStandardDatas(int,int,String)		100%		n/a	0 1	0 8	0 1
StandardDataController(StandardDataService)		100%		n/a	0 1	0 3	0 1
Total	0 of 42	100%	0 of 0	n/a	0 2	0 11	0 2

Created with JaCoCo

StandardDataServiceImpl

- **Result screenshot**

StandardDataServiceImplUnitTests (com.example.software_management.Service.Impl)

1秒 668毫秒

UT_STANDARD_DATA_SVC_001: 基本功能测试-默认参数

1秒 651毫秒

UT_STANDARD_DATA_SVC_002: 参数校正测试-页码为0

2毫秒

UT_STANDARD_DATA_SVC_003: 参数校正测试-pageSize为0

1毫秒

UT_STANDARD_DATA_SVC_004: 参数校正测试-pageSize超过最大值

2毫秒

UT_STANDARD_DATA_SVC_005: 搜索功能测试-有匹配结果

2毫秒

UT_STANDARD_DATA_SVC_006: 搜索功能测试-空白搜索词

2毫秒

UT_STANDARD_DATA_SVC_007: 仓库异常测试

8毫秒

[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.449 s - in com.example.software_management.Service.Impl.StandardDataServiceImplUnitTests

[INFO] Results:

[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

[INFO] Total time: 6.850 s

[INFO] Finished at: 2025-06-09T01:04:53+08:00

- **Results Table**

Test number	Test content	Target coverage point	Test result
UT_STANDARD_DATA_SVC_001	Basic function test - default parameters	Normal execution path	PASS

UT_STANDARD_DATA_SVC_002	Parameter calibration test - page number is 0	page < 1 calibration branch	PASS
UT_STANDARD_DATA_SVC_003	Parameter correction test - pageSize is 0	pageSize < 1 correction branch	PASS
UT_STANDARD_DATA_SVC_004	Parameter calibration test - pageSize exceeds the maximum value	pageSize > 50 correction branch	PASS
UT_STANDARD_DATA_SVC_005	Search function test - matching results	searchQuery non - empty branch	PASS
UT_STANDARD_DATA_SVC_006	Search function test - blank search term	searchQuery.trim() is empty branch	PASS
UT_STANDARD_DATA_SVC_007	Warehouse exception test	Exception handling	PASS

- **Test coverage description**
 - Instruction coverage reached 100%, covering all 125 instructions
 - The branch coverage also reaches 100%, covering all 12 branches
 - All methods (2 in total) are fully tested
 - The line coverage is 30 lines, fully covered

Software_Management > com.example.software_management.Service.Impl > StandardDataServiceImpl

StandardDataServiceImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
getStandardDatas(int, int, String)		100%		100%	0	7	0	27	0	1
StandardDataServiceImpl(StandardDataRepository)		100%		n/a	0	1	0	3	0	1
Total	0 of 125	100%	0 of 12	100%	0	8	0	30	0	2

9.2.1.2 Integrated Test

StandardDataController + StandardDataServiceImpl

- **Result screenshot**

✓ StandardDataControllerIntegrationTests (com.example.software_management.Controller)	1秒 639毫秒
✓ IT_STANDARD_DATA_CTRL_001: 基本功能测试-默认参数	1秒 613毫秒
✓ IT_STANDARD_DATA_CTRL_002: 分页参数测试-有效参数	2毫秒
✓ IT_STANDARD_DATA_CTRL_003: 参数校正测试-页码为0	1毫秒
✓ IT_STANDARD_DATA_CTRL_004: 参数校正测试- pageSize为0	2毫秒
✓ IT_STANDARD_DATA_CTRL_005: 参数校正测试- pageSize超过最大值	2毫秒
✓ IT_STANDARD_DATA_CTRL_006: 搜索功能测试-有匹配结果	4毫秒
✓ IT_STANDARD_DATA_CTRL_007: 搜索功能测试-无匹配结果	3毫秒
✓ IT_STANDARD_DATA_CTRL_008: 搜索边界测试-空搜索词	3毫秒
✓ IT_STANDARD_DATA_CTRL_009: 异常测试-Repository异常	7毫秒
✓ IT_STANDARD_DATA_CTRL_010: 组合测试-多参数校正	2毫秒

• Results Table

Test number	Test content	Target coverage point	Test result
IT_STANDARD_DATA_CTRL_001	Basic function test - default parameters	Full execution path	PASS
IT_STANDARD_DATA_CTRL_002	Paging parameter test - valid parameter	Paging function integrity	PASS
IT_STANDARD_DATA_CTRL_003	Parameter calibration test - page number is 0	page < 1 calibration branch	PASS
IT_STANDARD_DATA_CTRL_004	Parameter correction test - pageSize is 0	pageSize < 1 correction branch	PASS
IT_STANDARD_DATA_CTRL_005	Parameter calibration test - pageSize exceeds the maximum value	pageSize > 50 correction branch	PASS
IT_STANDARD_DATA_CTRL_006	Search function test - matching results	Search Function Integrity	PASS
IT_STANDARD_DATA_CTRL_007	Search function test - no matching results	Search no results handling	PASS
IT_STANDARD_DATA_CTRL_008	Search boundary test - empty search term	Empty Search Term Handling	PASS
IT_STANDARD_DATA_CTRL_009	Exception Test - Repository Exception	Exception handling branch	PASS
IT_STANDARD_DATA_CTRL_010	Combined Test - Multi-parameter Calibration	Parameter correction combination	PASS

9.2.2 Simulation function (API for obtaining simulation results)

9.2.2.1 Unit test

SimulationController

- Result screenshot

```
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.software_management.Controller.SimulationControllerUnitTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.695 s - in com.example.software_management.Controller.SimulationControllerUnitTests
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.11:report (default-report) @ Backend ---
[INFO] Loading execution data file F:\AAAFourthGrade\SoftwareManagement\TJ-Software-management\Codes\Backend\target\jacoco.exec
[INFO] Analyzed bundle 'Software_Management' with 49 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 20.142 s
[INFO] Finished at: 2025-06-09T00:34:54+08:00
[INFO] -----
```

✓ SimulationControllerUnitTests (com.example.software_management.Controller)	1秒 211毫秒
✓ UT_SIM_CTRL_001: 基本功能测试-正常响应	1秒 203毫秒
✓ UT_SIM_CTRL_002: 异常测试-服务异常	6毫秒
✓ UT_SIM_CTRL_003: 边界测试-文件为空	2毫秒

- Results Table

Test number	Test content	Target coverage point	Test result
UT_SIM_CTRL_001	Basic function test - normal response	Normal call service path	PASS
UT_SIM_CTRL_002	Abnormal test - service exception	Exception handling branch	PASS
UT_SIM_CTRL_003	Boundary test - File is empty	File verification error handling	PASS

- Test coverage description

- Instruction coverage is 100%, all 38 instructions have been covered
- Both methods have been fully tested
- Line coverage is 10 lines, covering all code

SimulationController

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
getSimulationResult(int,int,MultipartFile)		100%		n/a	0	1	0	7	0	1
SimulationController(SimulationService)		100%		n/a	0	1	0	3	0	1
Total	0 of 38	100%	0 of 0	n/a	0	2	0	10	0	2

SimulationServiceImplImplementation

- Result screenshot

```
Data set: HPT_eff_mod=0.1, Nf=0.2, SmFan=0.3, T24=0.4, Wf=0.5, T48=0.6, Nc=0.7, SmHPC=0.8
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.13 s - in com.example.software_management.Service.Impl.SimulationServiceImplUnitTests
[INFO] Results:
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco-maven-plugin:0.8.11:report (default-report) @ Backend ---
[INFO] Loading execution data file F:\AAAFourthGrade\SoftwareManagement\TJ-Software-management\Codes\Backend\target\jacoco.exec
[INFO] Analyzed bundle 'Software_Management' with 49 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 10.710 s
[INFO] Finished at: 2025-06-09T00:57:52+08:00
```

SimulationServiceImplUnitTests (com.example.software_management.Service.Impl)	2秒 565毫秒
UT_SIM_SERV_001: 标准数据验证测试-无效ID	2秒 91毫秒
UT_SIM_SERV_002: 标准数据验证测试-文件不存在	29毫秒
UT_SIM_SERV_003: 设备验证测试-无效ID	21毫秒
UT_SIM_SERV_004: 设备图片测试-无图片	43毫秒
UT_SIM_SERV_005: 设备图片测试-图片不存在	30毫秒
UT_SIM_SERV_006: 文件验证测试-空文件	33毫秒
UT_SIM_SERV_007: 文件验证测试-只有表头	46毫秒
UT_SIM_SERV_008: 文件验证测试-非CSV	25毫秒
UT_SIM_SERV_009: 文件验证测试-多行数据	17毫秒
UT_SIM_SERV_010: 文件验证测试-缺少列	19毫秒
UT_SIM_SERV_011: 数据验证测试-数据列不全	20毫秒
UT_SIM_SERV_012: 数据验证测试-非数值数据	13毫秒
UT_SIM_SERV_013: 数据验证测试-列不存在	14毫秒
UT_SIM_SERV_014: 不同设备类型测试-发动机	14毫秒
UT_SIM_SERV_015: 不同设备类型测试-风扇	12毫秒
UT_SIM_SERV_016: 异常测试-文件大小超限	44毫秒
UT_SIM_SERV_017: 测试保存预测结果	17毫秒
UT_SIM_SERV_018: 测试构建响应	13毫秒
UT_SIM_SERV_019: 测试保存数据文件方法	35毫秒
UT_SIM_SERV_020: 完整流程测试-成功场景	29毫秒

- Results Table

Test number	Test content	Target coverage point	Test result

UT_SIM_SERV_001	Standard Data Validation Test - Invalid ID	Invalid standard data ID verification	PASS
UT_SIM_SERV_002	Standard data validation test - file does not exist	Standard data file does not exist verification	PASS
UT_SIM_SERV_003	Device Verification Test - Invalid ID	Invalid device ID verification	PASS
UT_SIM_SERV_004	Equipment Picture Test - No Picture	Device no image URL processing	PASS
UT_SIM_SERV_005	Device picture test - picture does not exist	Device picture does not exist for processing	PASS
UT_SIM_SERV_006	File verification test - empty file	Empty file processing	PASS
UT_SIM_SERV_007	File verification test - header only	Table Header File Handling Only	PASS
UT_SIM_SERV_008	File verification test - non-CSV	File format verification	PASS
UT_SIM_SERV_009	File validation test - multi-line data	Multi-line data file validation	PASS
UT_SIM_SERV_010	File validation test - missing columns	Column inspection	PASS
UT_SIM_SERV_011	Data validation test - incomplete data column	Data integrity check	PASS
UT_SIM_SERV_012	Data validation test - non-numerical data	Non-numerical data processing	PASS
UT_SIM_SERV_013	Data validation test - column does not exist	Missing column processing	PASS
UT_SIM_SERV_014	Different equipment type testing - engine	Logic based on device name	PASS
UT_SIM_SERV_015	Testing of Different Equipment Types - Fan	Logic based on device name	PASS
UT_SIM_SERV_016	Exception test - file size exceeded limit	File size limit check	PASS
UT_SIM_SERV_017	Test save prediction results	Prediction result saving logic	PASS
UT_SIM_SERV_018	Test build response	Response build logic	PASS
UT_SIM_SERV_019	Test save data file method	Data storage logic	PASS

UT_SIM_SERV_020	Full Process Testing - Success Scenarios	Full execution path	PASS
-----------------	--	---------------------	------

• Test coverage description

- The overall instruction coverage rate is 84%, covering 136 of the 889 instructions
- The branch coverage rate is 62%, covering 24 of the 64 branches
- Tested 17 methods, but there is a lot of code that is not fully covered
- In particular `saveUploadedFile` method has only 37% instruction coverage and 66% branch coverage
- `processData` method coverage is low, with only 75% instruction coverage and 33% branch coverage
- The `createDirectory` and `init` methods have 0% coverage, indicating that the system was not tested to create the folder path
- The row coverage is 35 out of 187 rows, about 18.7%.

SimulationServiceImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
saveUploadedFile(MultipartFile)		37%		66%	2 4	6 10	0 1
processData(String, String)		75%		33%	5 7	7 22	0 1
createDirectoryIfNotExists(String)		0%		0%	3 3	6 6	1 1
saveDataFromFile(String, Component)		91%		75%	2 5	4 44	0 1
validateCsvFile(String)		87%		80%	2 6	5 22	0 1
init()		0%		n/a	1 1	4 4	1 1
saveForecastResult(Component, Model, Map, String)		83%		50%	3 4	3 14	0 1
setDoubleValueIfExists(Map, String[], String, Consumer)		100%		66%	2 4	0 14	0 1
getSimulationResult(int, int, MultipartFile)		100%		n/a	0 1	0 15	0 1
validateModel(int)		100%		100%	0 3	0 8	0 1
validateComponent(int)		100%		87%	1 5	0 11	0 1
buildResponse(String, Map, Forecast)		100%		n/a	0 1	0 7	0 1
SimulationServiceImpl(ModelRepository, ComponentRepository, ForecastRepository, DataRepository, RestTemplate)		100%		n/a	0 1	0 6	0 1
setUploadDir(String)		100%		n/a	0 1	0 1	0 1
setModelDir(String)		100%		n/a	0 1	0 1	0 1
setImageDir(String)		100%		n/a	0 1	0 1	0 1
setFlaskApiUrl(String)		100%		n/a	0 1	0 1	0 1
Total	136 of 889	84%	24 of 64	62%	21 49	35 187	2 17

9.2.2.2 Integrated Test

SimulationController + SimulationServiceImpl

• Result screenshot

✓	SimulationControllerIntegrationTests (com.example.software_management.Controller)	2秒 9毫秒
✓	IT_SIM_CTRL_001: 基本功能测试-正常输入	1秒 880毫秒
✓	IT_SIM_CTRL_002: 标准数据验证测试-无效ID	20毫秒
✓	IT_SIM_CTRL_003: 设备验证测试-无效ID	19毫秒
✓	IT_SIM_CTRL_004: 设备图片测试-无图片	19毫秒
✓	IT_SIM_CTRL_005: 文件验证测试-空文件	19毫秒
✓	IT_SIM_CTRL_006: 文件验证测试-非CSV	18毫秒
✓	IT_SIM_CTRL_007: 不同设备类型测试-发动机	17毫秒
✓	IT_SIM_CTRL_008: 不同设备类型测试-风扇	17毫秒

• Results Table

Test number	Test content	Target coverage point	Test result
EN_SIM_CTRL_001	Basic Function Test - Normal Input	Full execution path	PASS
EN_SIM_CTRL_002	Standard Data Validation Test - Invalid ID	Invalid standard data ID processing	PASS
EN_SIM_CTRL_003	Device Verification Test - Invalid ID	Invalid device ID handling	PASS
EN_SIM_CTRL_004	Equipment Picture Test - No Picture	Device has no image processing	PASS
EN_SIM_CTRL_005	File verification test - empty file	Empty file error handling	PASS
EN_SIM_CTRL_006	File verification test - non-CSV	File format error handling	PASS
EN_SIM_CTRL_007	Different equipment type testing - engine	Device type processing	PASS
EN_SIM_CTRL_008	Testing of Different Equipment Types - Fan	Device type processing	PASS

9.2.3 Statistical Chart



10. Summary

10.1 Overview of the testing process

The test work of the aero - engine life prediction system uses a comprehensive test method combining black box and white - box testing.

The test process focused on two key APIs: the standard data list API and the simulation API.

10.2 Overall test statistics

- Total number of test cases: 61 (37 Black box tests and 24 white-box tests)
- Pass rate: 92% (56 passed, 5 failed)
- Code coverage: There are significant differences among components (100% for controllers and standard data services, 84% for mock services)
- Integration test: 18 test cases, 100% pass rate

10.3 Key Findings

10.3.1 Black box testing

a. Parameter type processing problem :

- Non - integer input of paging parameters (page, pageSize) leads to 400 Bad Request error instead of graceful handling
- The system did not attempt to correct invalid input as expected by the test specification

b. HTTP status code inconsistency:

- For file size limit violations, the system returns 400 Bad Request instead of the standard 413 Payload Too Large
- All validation errors return the same status code, making it difficult to distinguish between different types of errors

c. Safe handling :

- The system correctly handled security-related test cases, including SQL injection and XSS attack attempts
- The input validation of search parameters is robust and can prevent malicious code execution

10.3.2 White-box testing

a. Code coverage advantage :

- 100% instruction and branch coverage for the controller and StandardDataService
- The error handling paths of all components are fully tested

b. Coverage gap:

- SimulationService shows significant coverage gaps (84% instruction coverage, 62% branch coverage)
- Key methods like `saveUploadedFile()` (37%), `processData()` (75%) and the initialization method (0%) lack sufficient coverage
- The exception handling scenario is well tested, but some business logic branches need more testing

c. Integration Test:

- The E2E Test verifies the correct interaction between the controller and the service
- All integration tests passed, confirming the correctness of the overall system functionality

10.4 Improvement Suggestions

10.4.1 Code Improvement

a. Enhanced parameter validation:

- Implement custom parameter binding in the controller to handle non - integer paging parameters
- Use `@InitBinder` to register a custom editor to convert a string to an integer with appropriate error handling

b. Standardized HTTP Status Code :

- Improve exception handling and use appropriate HTTP status codes for different error conditions
- Implement specific exception handlers in the global `@ControllerAdvice` class
- For file size violations, use 413 Payload Too Large as per HTTP standards

c. Improve the code coverage of `SimulationService`:

- Add test cases for the initialization method and directory creation logic
- Enhanced file handling test cases to cover `saveUploadedFile()` method in more branches
- Implement comprehensive testing for the `processData()` method to cover all device type scenarios

d. API documentation enhancement:

- Update API documentation to clearly specify parameter constraints and error responses
- Record all possible error status codes and their meanings
- Provide examples of valid and invalid requests

10.4.2 Test process improvement

a. Test case design :

- Align test expectations with the actual behavior of the system, or update the system to match the expected criteria
- Extended boundary value tests to cover more edge cases, especially file handling

b. Test coverage improvement :

- Add targeted tests for uncovered code paths in `SimulationService`

- Perform performance testing for file upload and processing functions
- Add more integration tests for complex workflows

c. Monitoring and continuous testing:

- Implement continuous integration with automated test execution
- Set code coverage thresholds as quality gates (minimum 90% statement coverage and 80% branch coverage)
- Add performance regression tests to ensure the system maintains response time standards

10.4.3 Architectural and design improvements

a. Error handling strategy:

- Implement a consistent error handling strategy across all controllers
- Create a hierarchical exception system to provide specific exception types for different error scenarios
- Use HTTP status codes according to RESTful API best practices

b. Input Validation Framework:

- Implement a centralized validation framework using Bean Validation (JSR 380)
- Use validation groups to separate different validation scenarios
- Add custom validators for complex business rules

c. Reconstruction suggestions :

- Extract file processing logic from SimulationService into a dedicated FileService
- Implement the strategy pattern for handling logic for different device types
- Consider using reactive programming for file processing to improve performance

Passing Criteria	Test Results	Met?
1. Standard data API handles pagination and search correctly	83.3% pass rate; issues with non-integer parameters	Partially
2. Simulation API verifies file format/structure, rejects invalid inputs	92.9% pass rate; properly validates most inputs	Yes
3. System generates accurate prediction results	Successfully produces life expectancy and health indices	Yes

4. Clear error messages with appropriate HTTP status codes	Issues with HTTP 413 for large files; technical errors for some parameters	Partially
5. Tests pass rate $\geq 90\%$, code coverage $\geq 80\%$	Black box: 87.5% overall pass rate; Coverage: 100% for data API, 84%/62% for simulation	Partially